

FlexTree: A Stateless Hash-Based Signature Scheme

Siwei Sun¹, siweisun.isaac@gmail.com

Mehdi Abri², m8abri@gmail.com

Jonathan Katz³, jkatz2@gmail.com

Zhen Qin¹, qinzhen23@mails.ucas.ac.cn

¹School of Cryptology, University of Chinese Academy of Sciences, China

²University of Isfahan, Iran

³Google, United States

June 29, 2026

Contents

Chapter 1 Specification	11
1.1 Notations and Preliminaries	11
1.1.1 Arrays, Byte Strings, and Integers	12
1.1.2 Merkle Trees	12
1.1.3 Encoding Functions	13
1.2 FlexTree Parameters	20
1.3 Cryptographic Functions	21
1.4 The FlexTree Hypertree	21
1.5 Address Scheme	24
1.6 Member Functions	26
1.7 The One-time Signature (OTS) Scheme	27
1.7.1 OTS Public Key Generation	28
1.7.2 OTS Signature Generation	28
1.7.3 Computing an OTS Public Key From a Signature	30
1.8 eXtended Merkle Signature Scheme (XMSS)	31
1.8.1 Generating a Merkle Hash Tree	32
1.8.2 Generating an XMSS Signature	32
1.8.3 Computing an XMSS Public Key from a XMSS Signature	32
1.9 The FlexTree Hypertree Signature	35
1.9.1 Hypertree Signature Generation	35
1.9.2 Hypertree Signature Verification	35
1.10 The PORS+FP Few-time Signature	36
1.10.1 Generating PORS+FP Secret Values	36
1.10.2 Computing the Nodes of a PORS Tree	37
1.10.3 Generating a PORS+FP Signature	38
1.10.4 Computing a PORS+FP Public Key from a Signature	38
1.11 The FlexTree Signature Scheme	40
1.11.1 FlexTree Key Generation	42
1.11.2 FlexTree Signature Generation	42
1.11.3 Signature Verification	44
1.12 Parameter Sets and Instantiations	44
Chapter 2 Design Rationale	49
2.1 Notations and Preliminaries	49
2.2 The Inhomogeneous WOTS ^c One-time Signature	50
2.3 The PORS+FP Few-time Signature Scheme	51
2.3.1 PORS: PRNG to Obtain a Random Subset	51

2.3.2 Limiting the Number of Authentication Nodes	51
2.4 The FlexTree Hypertree	52
Chapter 3 Security Analysis	55
3.1 Security Reduction	55
3.2 Concrete Security	56
Chapter 4 Performance Analysis	59
4.1 Theoretical Performance Analysis	59
4.2 Implementations and Benchmarks	61
4.3 Comparison with NIST Standard SLH-DSA	61
Chapter 5 Feature Overview	63
5.1 Techniques for Improving the SPHINCS⁺ Framework	63
5.2 Advantages and Limitations	64
Bibliography	64

List of Figures

1.1	A PORs tree with 12 leaves	14
1.2	A PORs tree with 13 leaves	14
1.3	A PORs tree with 15 leaves	14
1.4	FlexTree hypertree	23
1.5	OTS hash address	24
1.6	OTS secret key generation address	24
1.7	OTS public key compression address	25
1.8	XMSS tree hash address	25
1.9	FTS tree hash address	25
1.10	FTS secret key generation address	26
1.11	Root hash address	26
1.12	Inhomogeneous WOTS ^C signature data format	29
1.13	XMSS signature data format for the i -th layer	31
1.14	Hypertree signature data format	35
1.15	PORS+FP signature data format	37
1.16	FlexTree signature data format	43
2.1	The set of authentication nodes for $\mathcal{I} = \{1, 6, 7, 8\}$	52
2.2	The set of authentication nodes for $\mathcal{I} = \{4, 5, 6, 7\}$	52

List of Tables

1.1	Member functions for addresses	27
1.2	Member functions for compressed addresses	27
1.3	FlexTree parameter sets	46
1.4	Definition of the functions for FlexTree-160 and FlexTree-256	47
1.5	Definition of the functions for FlexTree-384 and FlexTree-512	48
4.1	A theoretical analysis of the performance of the proposed signature algorithms	61
4.2	Benchmark results for the proposed signature algorithms	61
4.3	Performance comparison between SPHINCS⁺-SHAKE-256* and FlexTree-SHAKE-256*	62

List of Algorithms

1	<code>toInt(X, n)</code>	12
2	<code>toByte(x, n)</code>	13
3	$\rho_1^{(t)} : \left[\binom{t}{k} \right] \rightarrow \binom{[t]}{k}$	15
4	$\rho_2^{(t)} : \left[\binom{t}{k} \right] \rightarrow \binom{[t]}{k}$	16
5	$\rho^{(t)} : \left[\binom{t}{k} \right] \rightarrow \binom{[t]}{k}$	17
6	<code>LowerBound</code>	19
7	<code>chain($n, i, s, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	28
8	<code>pkGen_{OTS}($\mathbf{SK.seed}, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	29
9	<code>Sign_{OTS}($n, \mathbf{SK.seed}, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	30
10	<code>pkFromSig_{OTS}($\mathbf{SIG}_{\text{OTS}}, M, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	31
11	<code>xmssNode($\mathbf{SK.seed}, i, z, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	32
12	<code>Sign_{XMSS}($M, \mathbf{SK.seed}, \text{idx}, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	33
13	<code>pkFromSig_{XMSS}($\text{idx}, \mathbf{SIG}_{\text{XMSS}}, M, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	34
14	<code>sign_{HT}($M, \mathbf{SK.seed}, \mathbf{PK.seed}, \text{idxTree}, \text{idxLeaf}$)</code>	36
15	<code>verify_{HT}($M, \mathbf{SIG}_{\text{HT}}, \mathbf{PK.seed}, \text{idxTree}, \text{idxLeaf}, \mathbf{PK.root}$)</code>	37
16	<code>skGen_{PORS}($\mathbf{SK.seed}, \mathbf{PK.seed}, \mathbf{ADRS}, j$)</code>	38
17	<code>node_{PORS}($\mathbf{SK.seed}, i, z, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	39
18	<code>octopus($\mathcal{I}$)</code>	39
19	<code>sign_{FTS}($\text{md}, \mathbf{SK.seed}, \mathbf{PK.seed}, \mathbf{ADRS}, \mathbf{R}$)</code>	40
20	<code>pkFromSig_{PORS}($\mathbf{SIG}_{\text{PORS}}, \text{md}, \mathbf{PK.seed}, \mathbf{ADRS}, \mathbf{R}$)</code>	40
21	<code>computeRoot($\sigma'_{\text{PORS}}, \mathcal{I}, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	41
22	<code>keyGen_{SLH}($\mathbf{SK.seed}, \mathbf{SK.prf}, \mathbf{PK.seed}$)</code>	42
23	<code>signInternal_{SLH}($\text{msg}, \mathbf{SK}, \text{addrnd}$)</code>	43
24	<code>verifyInternal_{SLH}($\text{msg}, \mathbf{SIG}, \mathbf{PK}$)</code>	44

Chapter 1

Specification

This chapter specifies the stateless hash-based signature scheme **FlexTree**, and provides instances meeting the security requirement made by the *Call for Proposals for the Next-generation Public-Key Cryptographic Algorithms* from the Institute of Commercial Cryptography Standards of China (NICCS) [1]. The call for proposal requires that the submitted algorithms should provide parameter sets achieving the following security levels:

- Category I: 128-bit classical security and 80-bit quantum security.
- Category II: 256-bit classical security and 128-bit quantum security.
- Category III: 512-bit classical security and 256-bit quantum security.

Parameter sets with 384-bit classical security and 192-bit quantum security is optional. In this chapter, we provide parameter sets for all the security levels listed above, including the optional one. Note that for category I, our parameter set achieves 160-bit classical security and 80-bit quantum security.

1.1 Notations and Preliminaries

For a finite set $\mathbb{G}$, the number of elements in $\mathbb{G}$ is denoted by $|\mathbb{G}|$. We use $\mathbb{Z}_+$ to denote the set of all positive integers and let $\mathbb{N} = \mathbb{Z}_+ \cup \{0\}$. The set of real numbers is denoted by $\mathbb{R}$. Let $\mathbb{F}_2 = \{0, 1\}$ be the binary field and $\mathbb{B} = \mathbb{F}_2^8$ be the set of all 8-bit strings (bytes). For a positive integer w , $[w]$ is defined to be the set $\{0, \dots, w-1\}$. The Cartesian product of two sets A and B is represented by $A \times B$. For $\mathbf{w} = (w_0, \dots, w_{l-1}) \in \mathbb{Z}_+^l$, the set $[w_0] \times \dots \times [w_{l-1}]$ is abbreviated as $\prod_{j=0}^{l-1} [w_j]$. When $w_0 = \dots = w_{l-1} = w$, $\prod_{j=0}^{l-1} [w_j] = [w]^l$. For $i \in \mathbb{N}$ and $m \in \mathbb{Z}_+$, $\text{toBin}(i, m)$ is the m -bit binary representation of i . For example, $\text{toBin}(5, 8) = 00000101$. Let x be a binary string, $\text{Int}(x)$ denotes the unsigned integer it represents, $\text{leadingBits}(x, n)$ denotes the most significant n bits and $\text{trailingBits}(x, n)$ denotes the least significant n bits of x .

To simplify the presentation, we introduce the helper function $\text{IntSeq}_{\mathbf{w}}(\cdot)$, which converts a binary string or an integer to a sequence of integers. Let $\mathbf{w} = (w_0, \dots, w_{v-1}) \in \mathbb{Z}_+^v$ and s be a $(\log_2 w_0 + \dots + \log_2 w_{v-1})$ -bit binary string. Then, $\text{IntSeq}_{\mathbf{w}}(s) = (i_0, \dots, i_{v-1})$, where i_t for $0 \leq t < v$ is the non-negative integer represented by the $\log_2 w_t$ -bit substring formed by the $((\sum_{j=0}^t \log_2 w_j) - \log_2 w_t)$ -th bit to $((\sum_{j=0}^t \log_2 w_j) - 1)$ -th bit segment of s . When the input is a non-negative integer $k \in \{0, \dots, 2^{\log_2 w_0 + \dots + \log_2 w_{v-1}} - 1\}$, $\text{IntSeq}_{\mathbf{w}}(k)$ should be understood as $\text{IntSeq}_{\mathbf{w}}(\text{toBin}(k, \log_2 w_0 + \dots + \log_2 w_{v-1}))$.

Example 1. Let $s = 101011$ be a 6-bit binary string, $k = 27$ be an integer whose binary representation is 11011 , and $\mathbf{w} = (4, 8, 2)$. Then, $\text{IntSeq}_{\mathbf{w}}(s) = (2, 5, 1)$, and $\text{IntSeq}_{\mathbf{w}}(k) = \text{IntSeq}_{\mathbf{w}}(\text{toBin}(k, 6)) = \text{IntSeq}_{\mathbf{w}}(011011) = (1, 5, 1)$.

For $a \in \mathbb{N}$ and $b \in \mathbb{Z}_+$, $a \bmod b$ denotes the integer i in $\{0, \dots, b-1\}$ such that $a - i$ is divisible by b . In addition, $a \text{ quo } b$ denotes the integer $j = \frac{a-i}{b}$.

Let $l, w_0, \dots, w_{l-1}$ be positive integers. For $s \in [1 + \sum_{j=0}^{l-1}(w_j - 1)]$, we define $\mathcal{C}_s(\prod_{j=0}^{l-1}[w_j])$ to be the set $\{(v_0, \dots, v_{l-1}) \in \prod_{j=0}^{l-1}[w_j] : \sum_{j=0}^{l-1} v_j = s\}$.

For positive integers t and k with $k \leq t$, we define $\binom{[t]}{k}$ to be the set

$$\{(y_0, \dots, y_{k-1}) : y_0 < \dots < y_{k-1}, y_j \in [t], j \in [k]\}. \quad (1.1)$$

We regard $\binom{[t]}{k}$ to be the set of all k -element subsets of $[t]$ when necessary.

1.1.1 Arrays, Byte Strings, and Integers

If X is a byte string of length n , then $X[i]$ for $i \in \{0, \dots, n-1\}$ will refer to the i -th element in the string X . If X is an array of m n -byte strings, then $X[i]$ for $i \in \{0, \dots, m-1\}$ will refer to the i -th n -byte string in X , and X will refer to the $m \cdot n$ -byte string $X[0] \parallel X[1] \parallel \dots \parallel X[m-1]$. A byte string may be interpreted as the big-endian representation of an integer. In such cases, a byte string X of length n is converted to the integer

$$X[0] \cdot 256^{n-1} + X[1] \cdot 256^{n-2} + \dots + X[n-2] \cdot 256 + X[n-1]. \quad (1.2)$$

Similarly, an integer x may be converted to a byte string of length n by finding coefficients $x_0, x_1, \dots, x_{n-1} \in \{0, \dots, 255\}$ such that

$$x = x_0 \cdot 256^{n-1} + x_1 \cdot 256^{n-2} + \dots + x_{n-2} \cdot 256 + x_{n-1} \quad (1.3)$$

and then setting the byte string to be $x_0 \parallel \dots \parallel x_{n-1}$. Algorithm 1 is a function that converts a byte string X of length n to an integer, and Algorithm 2 is a function that converts an integer x to a byte string of length n .

Algorithm 1: $\text{toInt}(X, n)$

Input: An n -byte string X

Output: Integer value of X

```

1  $total \leftarrow 0$ 
2 for  $i$  from 0 to  $n-1$  do
3    $total \leftarrow 256 \cdot total + X[i]$ 
4 return  $total$ 
```

1.1.2 Merkle Trees

Our design extensively employs a special type of binary tree called Merkle trees. We assume the reader is familiar with binary trees and just fix some notations here.

Algorithm 2: toByte(x, n)**Input:** In integers x and n **Output:** An array of n bytes containing binary representation of x in big-endian byte-order

```

1 total  $\leftarrow x$ 
2 for  $i$  from 0 to  $n - 1$  do
3    $S[n - 1 - i] \leftarrow total \bmod 256$ 
4    $total \leftarrow (total \gg 8)$ 
5 return  $S$ 

```

Definition 1. A full binary tree is a binary tree with either 0 or 2 child nodes for each node. A complete binary tree is a binary tree where all levels are completely filled except the lowest, which is filled from left to right.

A Merkle tree is a type of full and complete binary tree whose interior nodes (non-leaf nodes) can be computed from their child nodes with some hash functions. In our design, the public key of a PORS+FP few-time signature scheme corresponds to the root of a Merkle tree with t leaves, called a PORS tree (see Section 1.10). For example, Figure 1.1, Figure 1.2, and Figure 1.3 present three PORS trees with 12 leaves, 13 leaves and 15 leaves, respectively. For a PORS tree with t leaves, the height of the tree is $h_{\text{PORS}} = \lceil \log_2 t \rceil$. Let $s = t - 2^{h_{\text{PORS}} - 1}$. There are $2s$ leaves with height 0 and $t - 2s$ leaves with height 1. Moreover, for $j \geq 2s$, the j -th leaf of the PORS tree is the $(j - s)$ -th leaf in the 1st level (the sequence of nodes with height 1) of the tree. Taking Figure 1.2 for example, the 10th leaf of the PORS tree is the $5 = (10 - 5)$ -th node in the 1st level. In our design, we also employ XMSS trees (see Section 1.8), which are perfect binary trees. A perfect binary tree is a type of full and complete binary tree with $2^{h'}$ nodes, where all interior nodes have exactly two children and all leaf nodes are at the same depth.

Lemma 1. The parent node of the j -th node in level i of a Merkle tree is the $\text{par}(j)$ -th node in level $i + 1$, where $\text{par}(j) = j \gg 1$.

Lemma 2. The i -th node and j -th node in a same level of a Merkle tree are siblings if and only if $i \oplus j = 1$. Therefore, let $\text{sib}(i) = i \oplus 1$. Then, i -th node and $\text{sib}(i)$ -th node in a same level are siblings.

Lemma 3. The j -th node in level i ($i < h$) of a Merkle tree with height h is a left node if and only if $j \wedge 1 = 0$.

Lemma 4. Given t leaf nodes $\mathbf{n}_0^{(0)}, \dots, \mathbf{n}_{t-1}^{(0)}$ it takes $t - 1$ hash evaluations to compute the root node of the Merkle tree whose leaf nodes are $\mathbf{n}_0^{(0)}, \dots, \mathbf{n}_{t-1}^{(0)}$.

Proof. The number of hash evaluations is the number of all interior (non-leaf) nodes, which is $s + 2^{h_{\text{PORS}} - 2 + 1} - 1 = t - 2^{h_{\text{PORS}} - 1} + 2^{h_{\text{PORS}} - 1} - 1 = t - 1$. $\square$

1.1.3 Encoding Functions

The PORS+FP few-time signatures in FlexTree (Section 1.10) uses a bijective encoding functions given by Algorithm 3.

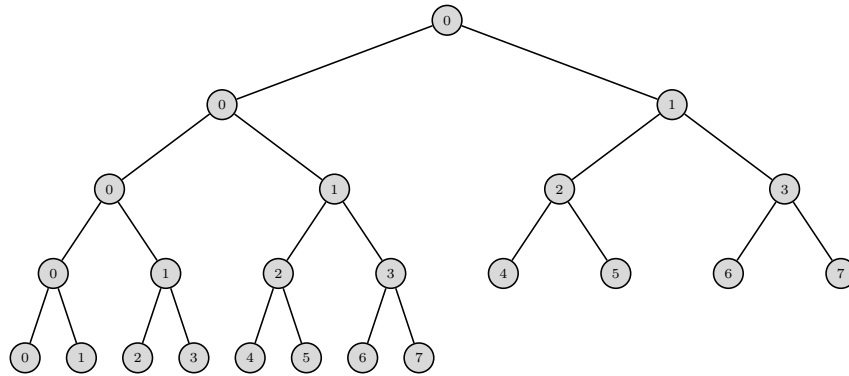


Figure 1.1: A PORs tree with 12 leaves

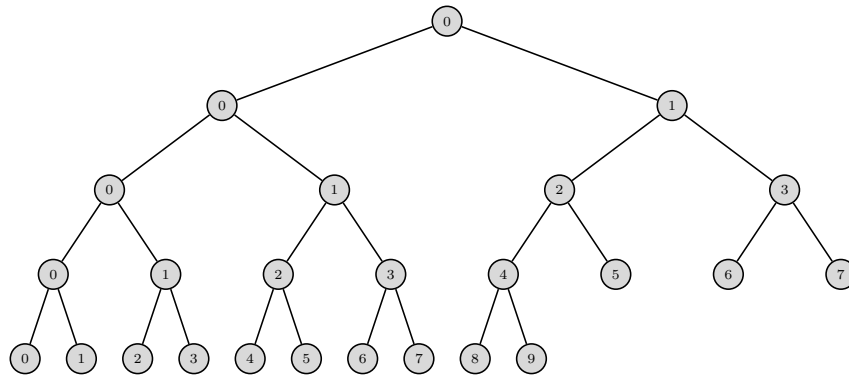


Figure 1.2: A PORs tree with 13 leaves

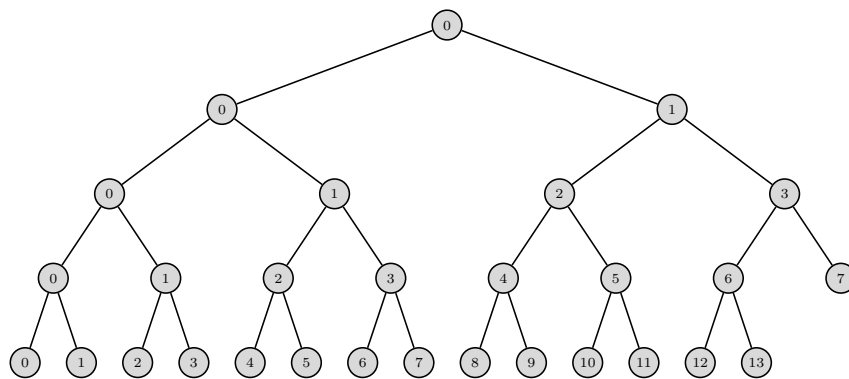


Figure 1.3: A PORs tree with 15 leaves

Lemma 5. Let $[t] = \{0, \dots, t-1\}$. Then, the map

$$\rho_1^{\binom{[t]}{k}} : \left[\binom{[t]}{k} \right] \rightarrow \binom{[t]}{k},$$

implemented by Algorithm 3 is a bijection.

Lemma 6. For $t > k$, we have the following identity

$$\binom{t}{k} = \binom{t-1}{k-1} + \binom{t-2}{k-1} + \dots + \binom{k-1}{k-1}. \quad (1.4)$$

For the integer $v \in \left[\binom{[t]}{k} \right]$, Algorithm 3 computes the k -element vector $\rho_1^{\binom{[t]}{k}}(v)$ entry by entry. If x is the value of y_{i-1} , then the remaining $k-i$ entries $(y_i, \dots, y_{k-1})$ must be chosen from $\{x+1, \dots, t-1\}$. Thus, there are $\binom{t-x-1}{k-i}$ possible completions.

Algorithm 3: $\rho_1^{\binom{[t]}{k}} : \left[\binom{[t]}{k} \right] \rightarrow \binom{[t]}{k}$

Input: An integer $v \in \left[\binom{[t]}{k} \right]$

Output: An increasing sequence $y = (y_0, \dots, y_{k-1})$ such that each $y_i \in [t]$

```

1   $r \leftarrow v$ 
2   $p \leftarrow 0$ 
3  for  $i$  from 1 to  $k$  do
4      for  $x$  from  $p$  to  $t - k + i - 1$  do
5           $c \leftarrow \binom{t-x-1}{k-i}$ 
6          if  $r \geq c$  then
7               $r \leftarrow r - c$ 
8          else
9               $y_{i-1} \leftarrow x$ 
10              $p \leftarrow x + 1$ 
11             break
12 return  $(y_0, \dots, y_{k-1})$ 

```

For efficiency considerations in the actual implementation, our code adopts a logic similar to that of Algorithm 5, instead of directly implementing Algorithm 3. We introduce Algorithm 4 to prove that Algorithm 5 is equivalent to Algorithm 3.

It is clear that, Algorithm 4 is equivalent to Algorithm 3. Consider the i -th iteration. The candidate value x in Algorithm 4 ranges over $x \in \{p, \dots, t - k + i - 1\}$. Instead of describing the selection process by the final position x , we describe it by the number of skipped blocks. Let $M = t - p - k + i$, for $0 \leq j \leq M - 1$, we define B_j recursively by $B_0 = 0$ and $B_{j+1} = B_j + \binom{t-p-j-1}{k-i}$, therefore $B_j = \sum_{\ell=0}^{j-1} \binom{t-p-\ell-1}{k-i}$. Here B_j denotes the total size of the blocks skipped before the j -th candidate is selected. The value $B_0 = 0$ is simply the initial value of this recurrence, corresponding to the situation where no block has yet been skipped.

The core of Algorithm 4 is to find the smallest $z \in \{p, \dots, t - k + i - 1\}$ such that

$$\binom{t-p-1}{k-i} + \binom{t-p-2}{k-i} + \dots + \binom{t-z}{k-i} + \binom{t-z-1}{k-i} > r. \quad (1.5)$$

Algorithm 4: $\rho_2^{\binom{[t]}{k}} : \binom{[t]}{k} \rightarrow \binom{[t]}{k}$

Input: An integer $v \in \binom{[t]}{k}$ **Output:** An increasing sequence $y = (y_0, \dots, y_{k-1})$ such that each $y_i \in [t]$

```

1  $r \leftarrow v$ 
2  $p \leftarrow 0$ 
3 for  $i$  from 1 to  $k$  do
4    $x \leftarrow \min \left\{ z \in \{p, \dots, t - k + i - 1\} : \sum_{u=p}^z \binom{t-u-1}{k-i} > r \right\}$ 
5   if  $x = p$  then
6      $r \leftarrow r$ 
7   else
8      $r \leftarrow r - \sum_{u=p}^{x-1} \binom{t-u-1}{k-i}$ 
9    $y_{i-1} \leftarrow x$ 
10   $p \leftarrow x + 1$ 
11 return  $(y_0, \dots, y_{k-1})$ 

```

Equivalently, it skips as many complete blocks as possible while the total skipped size does not exceed r . Hence, if $x = p + j^*$, then $j^* = \max\{j \in \{0, \dots, M-1\} : B_j \leq r\}$.

By identity $\binom{N}{r} - \binom{N-1}{r} = \binom{N-1}{r-1}$, taking $N = t - p - j$ and $r = k - i + 1$ gives $\binom{t-p-j-1}{k-i} = \binom{t-p-j}{k-i+1} - \binom{t-p-j-1}{k-i+1}$, substituting this into the expression of B_j , we obtain

$$\begin{aligned}
B_j &= \sum_{\ell=0}^{j-1} \left(\binom{t-p-\ell}{k-i+1} - \binom{t-p-\ell-1}{k-i+1} \right) \\
&= \left(\binom{t-p}{k-i+1} - \binom{t-p-1}{k-i+1} \right) + \left(\binom{t-p-1}{k-i+1} - \binom{t-p-2}{k-i+1} \right) \\
&\quad + \dots + \left(\binom{t-p-j+1}{k-i+1} - \binom{t-p-j}{k-i+1} \right).
\end{aligned}$$

All intermediate terms cancel out, and hence $B_j = \binom{t-p}{k-i+1} - \binom{t-p-j}{k-i+1}$. For $j = 0$, the same closed form also holds, since $\binom{t-p}{k-i+1} - \binom{t-p}{k-i+1} = 0 = B_0$. Therefore, for all $0 \leq j \leq M$, we have $B_j = \binom{t-p}{k-i+1} - \binom{t-p-j}{k-i+1}$.

Let $\text{base} = \binom{t-p}{k-i+1}$ and $\text{target} = \text{base} - r$. Then the condition $B_j \leq r$ is equivalent to $\binom{t-p}{k-i+1} - \binom{t-p-j}{k-i+1} \leq r$, and hence to $\binom{t-p-j}{k-i+1} \geq \text{target}$. We set $n = t - p - j$, as j ranges over $\{0, \dots, M-1\}$, the corresponding n ranges over $\{k-i+1, \dots, t-p\}$. Moreover, maximizing j is equivalent to minimizing n . Consequently, $n^* = t - p - j^* = \min\{n \in \{k-i+1, \dots, t-p\} : \binom{n}{k-i+1} \geq \text{target}\}$. This is exactly the value returned by `LowerBound`($k-i+1, t-p, k-i+1, \text{target}$). Finally, since $n^* = t - p - j^*$, we obtain $x = p + j^* = p + (t - p - n^*) = t - n^*$. Thus the assignment $x \leftarrow t - n$ in Algorithm 5 follows directly from the selection rule of Algorithm 4. When $k-i+1 = 1$, the same derivation gives $n^* = \min\{n \in \{1, \dots, t-p\} : n \geq \text{target}\} = \text{target}$, which explains the special branch $n \leftarrow \text{target}$ in Algorithm 5. Thus, we have Algorithm 4 is equivalent to Algorithm 5.

For the Algorithm 6, in each call to the lower-bound subroutine, we have $\text{low} = \text{choose}$ and $\text{high} = t - p$, where choose is the number of elements that still need to be selected

Algorithm 5: $\rho^{\binom{[t]}{k}} : \binom{[t]}{k} \rightarrow \binom{[t]}{k}$

Input: An integer $v \in \binom{[t]}{k}$ **Output:** An increasing sequence $y = (y_0, \dots, y_{k-1})$ such that each $y_i \in [t]$

```

1  $r \leftarrow v$ 
2  $p \leftarrow 0$ 
3 for  $i$  from 1 to  $k$  do
4    $\text{choose} \leftarrow k - i + 1$ 
5    $\text{low} \leftarrow \text{choose}$ 
6    $\text{high} \leftarrow t - p$ 
7    $\text{base} \leftarrow \text{GetPrecompBinom}(t - p, \text{choose})$ 
8    $\text{target} \leftarrow \text{base} - r$ 
9   if  $\text{choose} = 1$  then
10     $n \leftarrow \text{target}$ 
11     $c_n \leftarrow n$ 
12  else
13     $n, c_n \leftarrow \text{LowerBound}(\text{low}, \text{high}, \text{choose}, \text{target})$ 
14   $x \leftarrow t - n$ 
15   $r \leftarrow c_n - \text{target}$ 
16   $y_{i-1} \leftarrow x$ 
17   $p \leftarrow x + 1$ 
18 return  $(y_0, \dots, y_{k-1})$ 

```

and p is the smallest admissible value for the current element. Thus, it suffices to show that $t - p \geq \text{choose}$. Initially, $p = 0$ and $\text{choose} = k$, so the claim holds. Suppose that $t - p \geq \text{choose}$ holds at the beginning of an iteration. The algorithm selects an element x satisfying $x = t - n \leq t - \text{choose}$ and then sets $p' = x + 1$ and $\text{choose}' = \text{choose} - 1$. From $x \leq t - \text{choose}$, we get $t - p' = t - x - 1 \geq \text{choose} - 1 = \text{choose}'$. Hence the invariant is preserved. By induction, every call satisfies $\text{high} = t - p \geq \text{choose} = \text{low}$, so that $n \geq \text{choose}$ in Algorithm 6.

Some binomial coefficients used by the Algorithm 5 and 6 are precomputed before execution. The function $\text{GetPrecompBinom}(n, r)$ is used to obtain the value $\binom{n}{r}$. The parameter stride is a macro-defined constant that is fixed before the algorithm runs, which denotes the spacing between precomputed binomial coefficients. When $\text{stride} = 1$, the table contains every required value, so $\text{GetPrecompBinom}(n, r)$ can directly return $\binom{n}{r}$. When $\text{stride} > 1$, the Algorithm 6 first searches only over block boundary positions, namely multiples of stride . It computes $\text{firstBlock} = \lfloor \text{low} / \text{stride} \rfloor$ and $\text{lastBlock} = \lfloor \text{high} / \text{stride} \rfloor$, which are the first and last block indices whose boundary points lie in the interval $[\text{low}, \text{high}]$. If such block boundaries exist, the algorithm performs a binary search on the block indices. For a middle block midBlock , it sets $m = \text{midBlock} \cdot \text{stride}$ and compares $c_m = \binom{m}{\text{choose}}$ with the target T . If $c_m \geq T$, the desired point may be at this boundary or before it, so the search moves left; otherwise, the desired point must be after it, so the search moves right. If the desired point lies before the first block boundary, the binary search returns $\text{blockLow} = \text{firstBlock}$, and the later clipping step $\text{start} \leftarrow \max(\text{start}, \text{low})$ makes the final scan begin at low . If the desired point lies after the last block boundary, then no block boundary satisfies the condition, and the algorithm sets start to the position immediately after the last block

boundary and end to high. If there is no block boundary inside $[\text{low}, \text{high}]$, the algorithm simply scans the whole interval by setting $\text{start} = \text{low}$ and $\text{end} = \text{high}$. Finally, it computes $c = \binom{\text{start}}{\text{choose}}$ and scans forward until $c \geq T$, updating the binomial coefficient by the recurrence $c \leftarrow c \cdot (\text{start} + 1) / (\text{start} + 1 - \text{choose})$. Thus, this branch first performs a coarse search on block boundaries and then performs a fine scan inside the remaining small interval. A smaller stride gives more precomputed values and therefore less local scanning, but it requires more memory. A larger stride reduces memory usage, but it increases the maximum number of recurrence steps needed inside each block. Thus, stride controls the trade-off between storage cost and search time.

Line 43 of Algorithm 6 uses an exact-division optimization. Since the dividend is known to be divisible by the divisor, the power-of-two part is handled by right shifts, while the remaining odd part is divided by multiplying with its inverse modulo 2^{64} . This avoids expensive hardware division instructions and makes the division faster in practice.

Algorithm 6: LowerBound**Input:** Integers low, high, choose, and a target value T **Output:** The smallest integer $n \in [\text{low}, \text{high}]$ such that $\binom{n}{\text{choose}} \geq T$, together with $c_n = \binom{n}{\text{choose}}$

```

1 Notation
2 if stride = 1 then
3   while low < high do
4      $m \leftarrow \text{low} + \lfloor (\text{high} - \text{low})/2 \rfloor$ 
5      $c_m \leftarrow \text{GetPrecompBinom}(m, \text{choose})$ 
6     if  $c_m \geq T$  then
7       high  $\leftarrow m$ 
8     else
9       low  $\leftarrow m + 1$ 
10   $c_n \leftarrow \text{GetPrecompBinom}(\text{low}, \text{choose})$ 
11  return (low,  $c_n$ )
12 else
13  firstBlock  $\leftarrow \lceil \text{low}/\text{stride} \rceil$ 
14  lastBlock  $\leftarrow \lfloor \text{high}/\text{stride} \rfloor$ 
15  if firstBlock  $\leq$  lastBlock then
16    blockLow  $\leftarrow$  firstBlock
17    blockHigh  $\leftarrow$  lastBlock + 1
18    while blockLow < blockHigh do
19      midBlock  $\leftarrow$  blockLow +  $\lfloor (\text{blockHigh} - \text{blockLow})/2 \rfloor$ 
20       $m \leftarrow$  midBlock  $\cdot$  stride
21       $c_m \leftarrow \text{GetPrecompBinom}(m, \text{choose})$ 
22      if  $c_m \geq T$  then
23        blockHigh  $\leftarrow$  midBlock
24      else
25        blockLow  $\leftarrow$  midBlock + 1
26    if blockLow  $\leq$  lastBlock then
27      end  $\leftarrow$  blockLow  $\cdot$  stride
28      if blockLow = 0 then
29        start  $\leftarrow$  low
30      else
31        start  $\leftarrow$  (blockLow - 1)  $\cdot$  stride + 1
32    else
33      lastBase  $\leftarrow$  lastBlock  $\cdot$  stride
34      start  $\leftarrow$  lastBase + 1
35      end  $\leftarrow$  high
36    start  $\leftarrow$  max(start, low)
37    end  $\leftarrow$  min(end, high)
38  else
39    start  $\leftarrow$  low
40    end  $\leftarrow$  high
41   $c \leftarrow \text{GetPrecompBinom}(\text{start}, \text{choose})$ 
42  while start < end and  $c < T$  do
43     $c \leftarrow c \cdot (\text{start} + 1) / (\text{start} + 1 - \text{choose})$ 
44    start  $\leftarrow$  start + 1
45  return (start,  $c$ )

```

1.2 FlexTree Parameters

In our design, the following parameters are used.

- n : The security strength of the signature is $8n$ -bit with respect to classical attacks. The size of a node of an **XMSS** tree is n -byte. The node of a hash chain is n -byte. The size of a node of a **PORS** tree is n -byte.
- len : The inhomogeneous **WOTS^C** hash-chain based one-time signature scheme contains len chains.
- $\mathbf{w} = (w_0, \dots, w_{\text{len}-1})$: The generalized Winternitz parameter. For $0 \leq i < \text{len}$, the i -th chain of the one-time signature has w_i nodes, indexed by $\{0, \dots, w_i - 1\}$.
- z_b : When we sign the root node of an **XMSS** tree or a **PORS** tree with the inhomogeneous **WOTS^C** OTS, we need to repeatedly hash the node with a hash function until the highest z_b -bit segment of the output is 0^{z_b} .
- $n_{\text{ctr}}^{\text{OTS}}$: The size of the counter stored in the signature of an OTS in bytes. Currently, we set $n_{\text{ctr}}^{\text{OTS}} = 4$.
- t : A **PORS** tree has t leaf nodes.
- k : A **PORS+FP** signature contains the authentication information for k leaf nodes of the underlying **PORS** tree.
- m_{MAX} : A **PORS+FP** signature contains at most m_{MAX} authentication nodes, excluding the k leaves of the **PORS** tree to be authenticated.
- $n_{\text{ctr}}^{\text{FTS}}$: The **PORS+FP** signature contains a $n_{\text{ctr}}^{\text{FTS}}$ -byte counter. Currently we set $n_{\text{ctr}}^{\text{FTS}} = 4$.
- d : There are d layers of **XMSS** trees in the virtual hypertree.
- h : The total height of the **FlexTree** virtual hypertree is h .

When the above parameters are given, we can determine the values of the following parameters.

- $\mathbf{h} = (h_0, \dots, h_{d-1})$: The height of the **XMSS** trees in layer i of the virtual hypertree is h_i , where $h_0 = \dots = h_{k-1} = \lceil \frac{h}{d} \rceil$, $h_k = \dots = h_{d-1} = \lfloor \frac{h}{d} \rfloor$, and $h_0 + \dots + h_{d-1} = h$.
- m : To sign a message msg , we first hash it to an m -byte digest with

$$m = \left\lceil \frac{h - h_0}{8} \right\rceil + \left\lceil \frac{h_0}{8} \right\rceil + n.$$

- h_{PORS} : The height of the **PORS** tree is $h_{\text{PORS}} = \lceil \log_2 t \rceil$.
- s : Let $s = t - 2^{h_{\text{PORS}}-1}$. Then, there are $2s$ leaves with height 0 and $2^{h_{\text{PORS}}-1} - s$ leaves with height 1 in the **PORS** tree.

1.3 Cryptographic Functions

We employ the following functions in our design:

$$\left\{ \begin{array}{l} \mathbf{F} : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^n \rightarrow \mathbb{B}^n \\ \mathbf{H} : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{2n} \rightarrow \mathbb{B}^n \\ \mathbf{H}_{\text{PORS}} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^{n_{\text{ctr}}^{\text{FTS}}} \rightarrow \mathbb{B}^{n + \lceil \frac{\log_2(t)}{8} \rceil} \\ \mathbf{H}_{\text{Root}} : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^n \rightarrow \mathbb{B}^n \\ \mathbf{T}_{\text{len}} : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{\text{len} \cdot n} \rightarrow \mathbb{B}^n \\ \mathbf{PRF}_{\text{MSG}} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \rightarrow \mathbb{B}^n \\ \mathbf{H}_{\text{MSG}} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \rightarrow \mathbb{B}^m \\ \mathbf{PRF} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^{32} \rightarrow \mathbb{B}^n \end{array} \right. . \quad (1.6)$$

$\mathbf{F}$ is used to compute the nodes in the hash chains corresponding to the one-time signatures (see Line 5 of Algorithm 7) and the leaves of the PORS tree (see Line 6 and Line 11 of Algorithm 17). $\mathbf{H}$ is used to compute the non-leaf nodes of XMSS trees and PORS trees (see Line 11 of Algorithm 11 and Line 17 of Algorithm 17). $\mathbf{H}_{\text{PORS}}$ is used to compute an intermediate hash value during the signing process of the PORS+FP few-time signature (see Line 5 of Algorithm 19). $\mathbf{T}_{\text{len}}$ is used to compute the compression value of the end nodes of the hash chains for a one-time signature (see Line 13 of Algorithm 8), which is also a leaf node of an XMSS tree (see Line 4 of Algorithm 11). $\mathbf{H}_{\text{MSG}}$ is used to compute a digest for a message during the randomized Hash-and-Sign paradigm (see Line 4 of Algorithm 23). $\mathbf{PRF}_{\text{MSG}}$ is used to compute a random input value for $\mathbf{H}_{\text{MSG}}$ (see Line 3 of Algorithm 23). $\mathbf{PRF}$ is used to generate the secret preimages of the one-time signature (see Line 7 of Algorithm 8) and the secret preimages of the PORS+FP few-time signature (see Line 5 of Algorithm 16). $\mathbf{F}$, $\mathbf{H}$, $\mathbf{T}_{\text{len}}$, and $\mathbf{PRF}$ take a 32-byte data structure called address (the format of the address is specified in Section 1.5).

1.4 The FlexTree Hypertree

The FlexTree signature scheme can be described by a virtual hypertree as shown in Figure 1.4. The hypertree contains d layers. Layer 0 is the bottom layer and layer $d - 1$ is the top layer. In each layer there are certain number of perfect binary trees with the same height called XMSS trees. Let $h = h_0 + \dots + h_{d-1}$ be the total height of the hypertree. In layer 0, there are 2^{h-h_0} XMSS trees with height h_0 . In layer i , there are $2^{h-\sum_{u=0}^i h_u}$ XMSS trees with height h_i . In layer $d - 1$ (the top layer), there are exactly one XMSS tree with height h_{d-1} . Each XMSS tree with height h_i has 2^{h_i} leaves. Each leaf node corresponds to an instance of a one-time signature. Each leaf node in the bottom layer is associated with a PORS tree, and thus there are 2^h PORS trees. Each PORS tree has t leaves and is of height $\lceil \log_2 t \rceil$. In the hypertree, each XMSS tree in layer i is associated with a leaf node of an XMSS tree in layer $i + 1$. Specifically, for $0 \leq i < d - 1$, the j -th XMSS tree $\Phi_j^{(i)}$ in layer i is associated with the $(j \bmod 2^{h_{i+1}})$ -th leaf node of the $(j \text{ quo } 2^{h_{i+1}})$ -th XMSS tree in layer $i + 1$.

The secret key of FlexTree is $(\mathbf{SK.seed}, \mathbf{SK.prf}, \mathbf{PK.root}, \mathbf{PK.seed}) \in \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^n$, and the public key is $(\mathbf{PK.root}, \mathbf{PK.seed}) \in \mathbb{B}^n \times \mathbb{B}^n$, where $\mathbf{SK.seed}$ and $\mathbf{SK.prf}$ are private, $\mathbf{PK.root}$ is the root of the XMSS tree in the top layer, and $\mathbf{PK.seed}$ is public. To

sign a message $\text{msg} \in \mathbb{F}_2^*$, we first compute

$$\begin{cases} \mathbf{R} = \text{PRF}_{\text{MSG}}(\mathbf{SK}.\text{prf}, \text{optRand}, \text{msg}) \\ \text{digest} = \mathbf{H}_{\text{MSG}}(\mathbf{R}, \mathbf{PK}.\text{seed}, \mathbf{PK}.\text{root}, \text{msg}) \end{cases} ,$$

where optRand is $\mathbf{PK}.\text{seed}$ or a fresh random value. For the former case, **FlexTree** is a deterministic signature. Then, we divide the m -byte **digest** into three segments. The size of the 0th segment is $\lceil \frac{h-h'}{8} \rceil$ -byte, the size of the 1st segment is $\lceil \frac{h'}{8} \rceil$ -byte, and the size of the 2nd segment is n -byte. The 0th segment is used to select one **XMSS** tree in the bottom layer, and the 1st segment is used to select one leaf of the selected **XMSS** tree. Let md be the 2nd segment. Let the selected leaf node be the idxLeaf -th leaf of the idxTree -th **XMSS** tree in the bottom layer. Denote the j -th **XMSS** tree in layer i by $\Phi_j^{(i)}$. We first sign md with the **PORS+FP** instance associated with the t_0 -th leaf of the **XMSS** tree $\Phi_{j_0}^{(0)}$, where $j_0 = \text{idxTree}$, $t_0 = \text{idxLeaf}$. Next, we sign the public key of the **PORS+FP** instance with the OTS corresponding to the t_0 -th leaf node of $\Phi_{j_0}^{(0)}$. Then, we sign the root of $\Phi_{j_0}^{(0)}$ with the OTS corresponding to the t_1 -th leaf node of $\Phi_{j_1}^{(1)}$, $\dots$, and sign the root of $\Phi_{j_{d-3}}^{(d-3)}$ with the OTS corresponding to the t_{d-2} -th leaf node of $\Phi_{j_{d-2}}^{(d-2)}$. Finally, we sign the root of $\Phi_{j_{d-2}}^{(d-2)}$ with the OTS corresponding to the t_{d-1} -th leaf node of $\Phi_{j_{d-1}}^{(d-1)}$, where $j_{k+1} = j_k \bmod 2^{h_{k+1}}$, and $t_{k+1} = j_k \bmod 2^{h_{k+1}}$ ($0 \leq k < d$).

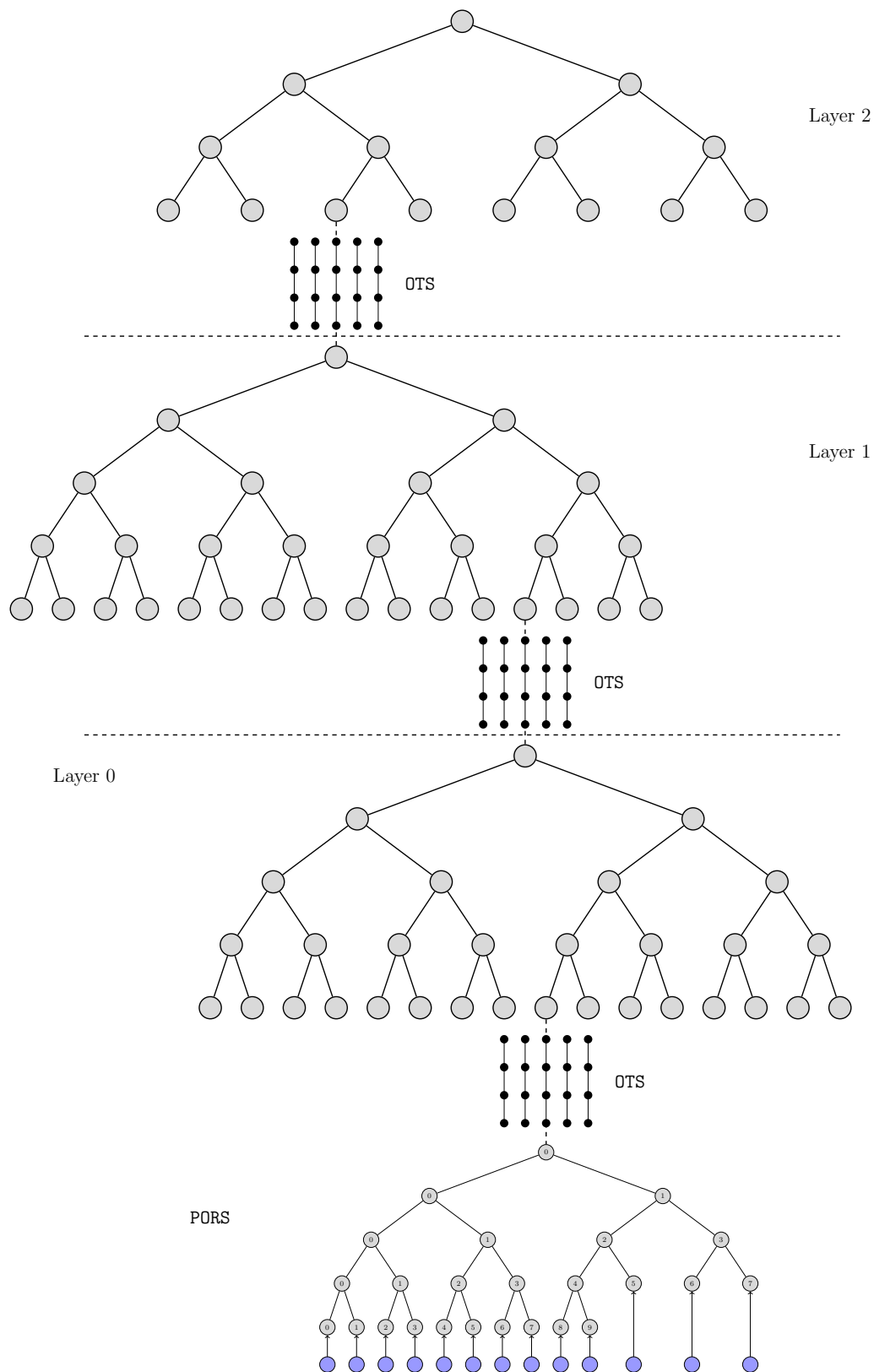


Figure 1.4: FlexTree hypertree

1.5 Address Scheme

The functions **F**, **H**, **H_{Root}**, **T_{len}** and **PRF** described in Section 1.3 take a 32-byte address ADRS as input. An ADRS consists of public values indicating the position of the data element within the virtual hypertree being computed by the functions. A different ADRS value is used for each call to each function. The structure of an ADRS conforms to word boundaries, with each word being 4 bytes long and the values of the data fields in ADRS are encoded as unsigned integers. There are 7 different types of addresses. These 7 types of ADRS share 3 common fields, including **layerAddr**, **treeAddr** and **addrType**. **layerAddr** $\in \{0, \dots, d-1\}$ takes 4 bytes, encoding the layer number of the data element being computed, with 0th layer being the bottom layer and $(d-1)$ -th layer being the top layer. **treeAddr** takes 12 bytes which indicates the index of the XMSS tree in a specific layer. If **layerAddr** = i for an address data structure, then the range of its **treeAddr** is $\{0, 1, \dots, 2^{h-\sum_{j=0}^i h_j} - 1\}$. **addrType** takes 4 bytes, indicating the type of the address, whose range is $\{\text{OTS_HASH} = 0x00, \text{OTS_PK} = 0x01, \text{XMSS_TREE} = 0x02, \text{FTS_TREE} = 0x03, \text{OTS_PRF} = 0x04, \text{FTS_PRF} = 0x05, \text{ROOT_HASH} = 0x06\}$, where **OTS_HASH** = 0x00 corresponds to an OTS hash address (see Figure 1.5), **OTS_PRF** = 0x04 corresponds to an OTS secret key generation address (see Figure 1.6), **OTS_PK** = 0x01 corresponds to an OTS public key compression address (see Figure 1.7), **XMSS_TREE** = 0x02 corresponds to an XMSS tree hash address (see Figure 1.8), **FTS_TREE** = 0x03 corresponds to an FTS tree hash address (see Figure 1.9), **FTS_PRF** = 0x05 corresponds to an FTS secret key generation address (see Figure 1.10), and **ROOT_HASH** = 0x06 corresponds to a root hash address (see Figure 1.11).

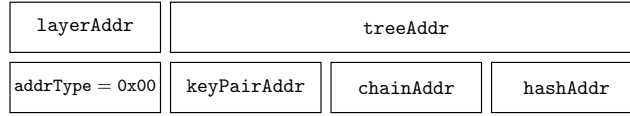


Figure 1.5: OTS hash address

As shown in Figure 1.5, **addrType** = **OTS_HASH** = 0x00 for an OTS hash address. An OTS hash address is used as input to **F** for computing the hash chains of OTS signatures (see Line 1 and Line 5 of Algorithm 7). Let **layerAddr** = i . The value of **keyPairAddr** indicates the index of the leaf of an XMSS tree with which the OTS instance concerned is associated, and its range is $\{0, 1, \dots, 2^{h_i} - 1\}$. The value of **chainAddr** determines the index of the hash chain being computed, whose range is $\{0, 1, \dots, \text{len} - 1\}$. Let **chainAddr** = j . **hashAddr** indicated the relative position of a node being computed within a specific hash chain, whose range is $\{0, 1, \dots, w_j - 2\}$.

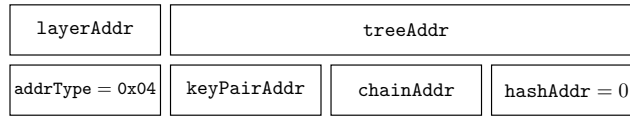


Figure 1.6: OTS secret key generation address

As shown in Figure 1.6, **addrType** = **OTS_PRF** = 0x04 for an OTS secret key generation address. **PRF** takes an OTS secret key generation address as input when computing the array of secret preimage elements of an OTS instance based on the secret seed **SK.seed** of the **FlexTree** secret key (see Line 3 and Line 7 of Algorithm 8). Let **layerAddr** = i . For an OTS secret key generation address, the value of **keyPairAddr** indicates the index of the

leaf of an XMSS tree with which the OTS instance concerned is associated, whose range is $\{0, 1, \dots, 2^{h_i} - 1\}$. The value of `chainAddr` determines the index of the hash chain being computed, whose range is $\{0, 1, \dots, \text{len} - 1\}$. The value of `hashAddr` is always set to 0 for an OTS secret key generation address.

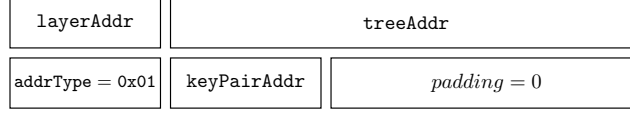


Figure 1.7: OTS public key compression address

As shown in Figure 1.7, `addrType` = OTS.PK = 0x01 for an OTS public key compression address. $\mathbf{T}_{\text{len}}$ takes an OTS public key compression address as input when compressing the end nodes of the hash chains of an OTS instance to an n -byte value, which corresponds to a leaf of an XMSS tree (see Line 11 and Line 13 of Algorithm 8). Let `layerAddr` = i . For an OTS public key compression address, the value of `keyPairAddr` indicates the index of the leaf of an XMSS tree with which the OTS instance concerned is associated, whose range is $\{0, 1, \dots, 2^{h_i} - 1\}$. The last 8 bytes of the address `ADRS` are set to 0.

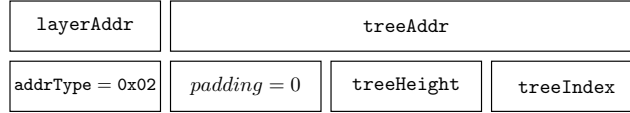


Figure 1.8: XMSS tree hash address

As shown in Figure 1.8, `addrType` = XMSS.TREE = 0x02 for an XMSS tree hash address. To construct the XMSS trees in the FlexTree hypertree, $\mathbf{H}$ takes an XMSS tree hash address as input (see Line 8 and Line 11 of Algorithm 11). Let `layerAddr` = i . For an XMSS tree hash address, the value of `treeHeight` indicates the height of the nodes being computed by $\mathbf{H}$ within an XMSS tree, whose range is $\{1, \dots, h_i\}$. The height of the root node is h_i . The nodes with height 0 (i.e., the leaves) are not computed by $\mathbf{H}$. Instead, they are computed with $\mathbf{F}$. The value of `treeIndex` determines the position of the node being computed within the h -th level (formed by the nodes with height h) of an XMSS tree, whose range is $\{0, \dots, 2^{h_i-h} - 1\}$.

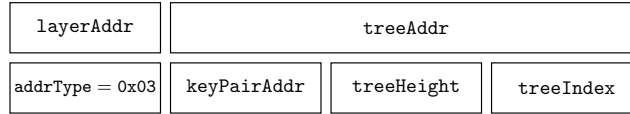


Figure 1.9: FTS tree hash address

As shown in Figure 1.9, `addrType` = FTS.TREE = 0x03 for an FTS tree hash address. $\mathbf{H}$ or $\mathbf{F}$ takes an FTS tree hash address as input to compute the nodes of an FTS tree, where $\mathbf{H}$ is used to compute the interior nodes and $\mathbf{F}$ is used to compute the leaf nodes (see Algorithm 17). For an FTS tree hash address, `layerAddr` = 0, `treeAddr` $\in \{0, \dots, 2^{h-h_0} - 1\}$, and the value of `keyPairAddr` indicates the index of the leaf of an XMSS tree with which the FTS instance concerned is associated, whose range is $\{0, 1, \dots, 2^{h_0} - 1\}$. The value of `treeHeight` indicates the height of the node being computed within an FTS tree, whose range is $\{0, \dots, h_{\text{PORS}}\}$. Note that the height of the root node is h_{PORS} and the nodes with

height 0 are leaf nodes. Let $\text{treeHeight} = h$. The value of treeIndex indicates the position of the node being computed within a specific level, whose range is $\{0, \dots, 2^{h_{\text{PORS}} - h} - 1\}$ for $h > 0$ and $\{0, \dots, 2s - 1\}$ for $h = 0$.

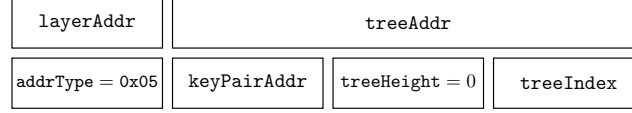


Figure 1.10: FTS secret key generation address

As shown in Figure 1.10, $\text{addrType} = \text{FTS_PRF} = 0x05$ for an FTS secret key generation address. **PRF** takes an FTS secret key generation address as input to compute the secret values of an FTS instance (see Line 5 of Algorithm 16). For an FTS secret key generation address, $\text{layerAddr} = 0$, $\text{treeAddr} \in \{0, \dots, 2^{h-h_0} - 1\}$, and $\text{treeHeight} = 0$. The value of keyPairAddr indicates the index of the leaf of an XMSS tree with which the FTS instance concerned is associated, whose range is $\{0, 1, \dots, 2^{h_0} - 1\}$. The value of treeIndex indicates the index of the secret value being computed, whose range is $\{0, \dots, 2^{h_{\text{PORS}}} - 1\}$.

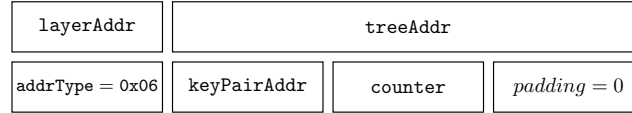


Figure 1.11: Root hash address

As shown in Figure 1.11, $\text{addrType} = \text{ROOT_HASH} = 0x06$ for a root hash address. $\mathbf{H}_{\text{Root}}$ takes a root hash address as input when computing a digest of the root of an XMSS tree or FTS tree during the signing operation of a one-time signature. Let $\text{layerAddr} = i$. For a root hash address, the value of keyPairAddr indicates the index of the leaf of a XMSS tree with which the OTS instance concerned is associated, whose range is $\{0, 1, \dots, 2^{h_i} - 1\}$. **counter** is initialized to 0 and incremented when $\mathbf{H}_{\text{Root}}$ is repeatedly applied.

1.6 Member Functions

If a complex data structure (e.g., an ADRS) contains a component $\mathbf{X}$, then $\text{ADRS.getX}()$ returns the value of $\mathbf{X}$, and $\text{ADRS.setX}(v)$ sets the component $\mathbf{X}$ in ADRS to the value held by v . If a data structure s contains multiple instances of $\mathbf{X}$, then $s.\text{getX}(i)$ returns the value of the i -th instance of $\mathbf{X}$ in s . Whenever the type in an address changes, the final 12 bytes of the address are initialized to zero. The member function $\text{ADRS.setTypeAndClear}(Z)$ for addresses sets the type of the ADRS to Z and sets the final 12 bytes of the ADRS to zero. Table 1.1 shows the member functions that operates on addresses.

The functions employed in **FlexTree-160** and **FlexTree-256** make use of a compressed version of ADRS. A compressed address (ADRS^c) is a 22-byte string that is the same as an ADRS with the exceptions that the encodings of the layer address and type are reduced to one byte each and the encoding of the tree address is reduced to eight bytes (i.e., $\text{ADRS}^c = \text{ADRS}[3] \parallel \text{ADRS}[8 : 16] \parallel \text{ADRS}[19] \parallel \text{ADRS}[20 : 32]$). The member functions for compressed addresses are shown in Table 1.2.

Table 1.1: Member functions for addresses

Member function	Functionality
$\text{ADRS.setLayerAddress}(i)$	$\text{ADRS} \leftarrow \text{toByte}(i, 4) \parallel \text{ADRS}[4 : 32]$
$\text{ADRS.setTreeAddress}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 4] \parallel \text{toByte}(i, 12) \parallel \text{ADRS}[16 : 32]$
$\text{ADRS.setTypeAndClear}(Y)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 16] \parallel \text{toByte}(Y, 4) \parallel \text{toByte}(0, 12)$
$\text{ADRS.setKeyPairAddress}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 20] \parallel \text{toByte}(i, 4) \parallel \text{ADRS}[24 : 32]$
$\text{ADRS.setChainAddress}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 24] \parallel \text{toByte}(i, 4) \parallel \text{ADRS}[28 : 32]$
$\text{ADRS.setTreeHeight}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 24] \parallel \text{toByte}(i, 4) \parallel \text{ADRS}[28 : 32]$
$\text{ADRS.setHashAddress}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 28] \parallel \text{toByte}(i, 4)$
$\text{ADRS.setTreeIndex}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 28] \parallel \text{toByte}(i, 4)$
$\text{ADRS.setCounter}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 28] \parallel \text{toByte}(i, 4) \parallel \text{ADRS}[28 : 32]$
$\text{ADRS.counterIncrement}()$	$\text{ADRS}[24 : 28] \leftarrow \text{toByte}(\text{toInt}(\text{ADRS}[24 : 28]) + 1, 4)$
$i \leftarrow \text{ADRS.getLayerAddress}()$	$i \leftarrow \text{toInt}(\text{ADRS}[0 : 4], 4)$
$i \leftarrow \text{ADRS.getKeyPairAddress}()$	$i \leftarrow \text{toInt}(\text{ADRS}[20 : 24], 4)$
$i \leftarrow \text{ADRS.getTreeIndex}()$	$i \leftarrow \text{toInt}(\text{ADRS}[28 : 32], 4)$
$i \leftarrow \text{ADRS.getCounter}()$	$i \leftarrow \text{toInt}(\text{ADRS}[24 : 28], 4)$

Table 1.2: Member functions for compressed addresses

Member function	Functionality
$\text{ADRS.setLayerAddress}(i)$	$\text{ADRS} \leftarrow \text{toByte}(i, 1) \parallel \text{ADRS}[1 : 22]$
$\text{ADRS.setTreeAddress}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 1] \parallel \text{toByte}(i, 8) \parallel \text{ADRS}[9 : 22]$
$\text{ADRS.setTypeAndClear}(Y)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 9] \parallel \text{toByte}(Y, 1) \parallel \text{toByte}(0, 12)$
$\text{ADRS.setKeyPairAddress}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 10] \parallel \text{toByte}(i, 4) \parallel \text{ADRS}[14 : 22]$
$\text{ADRS.setChainAddress}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 14] \parallel \text{toByte}(i, 4) \parallel \text{ADRS}[18 : 22]$
$\text{ADRS.setTreeHeight}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 14] \parallel \text{toByte}(i, 4) \parallel \text{ADRS}[18 : 22]$
$\text{ADRS.setHashAddress}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 18] \parallel \text{toByte}(i, 4)$
$\text{ADRS.setTreeIndex}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 18] \parallel \text{toByte}(i, 4)$
$i \leftarrow \text{ADRS.getLayerAddress}()$	$i \leftarrow \text{toInt}(\text{ADRS}[0 : 4], 4)$
$\text{ADRS.setCounter}(i)$	$\text{ADRS} \leftarrow \text{ADRS}[0 : 14] \parallel \text{toByte}(i, 4) \parallel \text{ADRS}[18 : 22]$
$\text{ADRS.counterIncrement}()$	$\text{ADRS}[14 : 18] \leftarrow \text{toByte}(\text{toInt}(\text{ADRS}[14 : 18]) + 1, 4)$
$i \leftarrow \text{ADRS.getKeyPairAddress}()$	$i \leftarrow \text{toInt}(\text{ADRS}[10 : 14], 4)$
$i \leftarrow \text{ADRS.getTreeIndex}()$	$i \leftarrow \text{toInt}(\text{ADRS}[18 : 22], 4)$
$i \leftarrow \text{ADRS.getCounter}()$	$i \leftarrow \text{toInt}(\text{ADRS}[14 : 18], 4)$

1.7 The One-time Signature (OTS) Scheme

This section presents the inhomogeneous WOTS^c one-time signature scheme, which is an inhomogeneous variant [2] of WOTS^c [3]. Inhomogeneous WOTS^c uses four parameters, including n , z_b , len , and $\mathbf{w} = (w_0, \dots, w_{\text{len}-1})$. The security parameter n is the length in bytes of the messages that may be signed as well as the length of the private key elements, public key elements, and signature elements. The third parameter len indicates the number of hash chains as well as the number of n -byte private key elements in the inhomogeneous WOTS^c . $\mathbf{w} = (w_0, \dots, w_{\text{len}-1})$ is the generalized Winternitz parameter, indicating that there are w_j nodes in the j -th hash chain. When we sign the root of an XMSS or PORS tree with inhomogeneous WOTS^c , we need to repeatedly hash the root with a hash function until the highest z_b -bit segment of the output is 0^{z_b} and for the string $\mathbf{y} \in \mathbb{F}_2^{8n-z_b}$ formed by the least

significant $8n - z_b$ bits of the output,

$$\mathbf{x} = (x_0, \dots, x_{1\text{en}-1}) = \text{IntSeq}_{\mathbf{w}}(\mathbf{y}) \in \mathcal{C}_s \left(\prod_{j=0}^{l-1} [w_j] \right),$$

where $\mathbf{s} = \left\lfloor \frac{(w_0-1) + \dots + (w_{1\text{en}-1}-1)}{2} \right\rfloor$ and $\log_2 w_0 + \dots + \log_2 w_{1\text{en}-1} = 8n - z_b$.

An inhomogeneous WOTS^c private key consists of 1en secret n -byte values. In **FlexTree**, these are all generated from an n -byte secret seed **SK.seed** using **PRF**. Hash chains of different lengths are then created from the secret values using a chaining function, and the end nodes from each of the chains are hashed to derive the inhomogeneous WOTS^c public key. The hash chains are computed by the **chain** function given by Algorithm 7. The **chain** function takes an n -byte string **n** and integers s and i (where $i + s < w$) as input and returns the result of iterating a hash function **F** on the input s times, starting from an index of i . The **chain** function also requires as input **PK.seed**, which is part of the **FlexTree** public key, and an address **ADRS**. The type in **ADRS** must be set to **OTS_HASH**, and the layer address, tree address, key pair address, and chain address must be set to the address of the chain being computed. The **chain** function updates the hash address in **ADRS** with each iteration to specify the current position in the chain prior to **ADRS**'s use in **F**.

Algorithm 7: **chain**(**n**, i , s , **PK.seed**, **ADRS**)

Input: An n -byte element **n**, starting index i , number of steps s , the public seed **PK.seed**, and the address **ADRS**

Output: An n -byte element (a node of the hash chain)

```

1 ASSERT ADRS.addrType = OTS_Hash
2  $tmp \leftarrow \mathbf{n}$ 
3 for  $j$  from  $i$  to  $i + s - 1$  do
4    $\text{ADRS.setHashAddress}(j)$ 
5    $tmp \leftarrow \mathbf{F}(\mathbf{PK.seed}, \mathbf{ADRS}, tmp)$ 
6 return  $tmp$ 

```

1.7.1 OTS Public Key Generation

The $\text{pkGen}_{\text{OTS}}$ function given by Algorithm 8 generates the WOTS^c public keys. It takes **SK.seed** and **PK.seed** from the WOTS^c private key and an address as input. The type in the address **ADRS** must be set to **OTS_HASH**, and the layer address, tree address, and key pair address must encode the address of the WOTS^c public key to be generated. Lines 5 through 9 in Algorithm 8 generate the end nodes of the hash chains. For each of the 1en end nodes, the corresponding secret value is generated in line 7, and the **chain** function is called to compute the end node of the hash chain. Once the 1en end nodes are computed, they are compressed into a single n -byte value in lines 10 through 13.

1.7.2 OTS Signature Generation

An inhomogeneous WOTS^c signature is a $(n_{\text{ctr}}^{\text{OTS}} + 1\text{en} \cdot n)$ -byte string, containing an $n_{\text{ctr}}^{\text{OTS}}$ -byte counter and an array of 1en n -byte strings as shown in Figure 1.12. The Sign_{OTS} function

Algorithm 8: $\text{pkGen}_{\text{OTS}}(\mathbf{SK}.\text{seed}, \mathbf{PK}.\text{seed}, \text{ADRS})$ **Input:** Secret seed $\mathbf{SK}.\text{seed}$, public seed $\mathbf{PK}.\text{seed}$, the address ADRS **Output:** The n -byte public key of the OTS (a leaf node of an $\mathbf{XMSS}$ tree)

```

1 ASSERT  $\text{ADRS}.\text{addrType} = \text{OTS\_Hash}$ 
2  $\text{skADRS} \leftarrow \text{ADRS}$ 
3  $\text{skADRS}.\text{setTypeAndClear}(\text{OTS\_PRF})$ 
4  $\text{skADRS}.\text{setKeyPairAddress}(\text{ADRS}.\text{getKeyPairAddress}())$ 
5 for  $i$  from 0 to  $\text{len} - 1$  do
6    $\text{skADRS}.\text{setChainAddress}(i)$ 
7    $sk \leftarrow \text{PRF}(\mathbf{PK}.\text{seed}, \mathbf{SK}.\text{seed}, \text{skADRS})$ 
8    $\text{ADRS}.\text{setChainAddress}(i)$ 
9    $\text{tmp}[i] \leftarrow \text{chain}(sk, 0, w_i - 1, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
10  $\text{otspkADRS} \leftarrow \text{ADRS}$ 
11  $\text{otspkADRS}.\text{setTypeAndClear}(\text{OTS\_PK})$ 
12  $\text{otspkADRS}.\text{setKeyPairAddress}(\text{ADRS}.\text{getKeyPairAddress}())$ 
13  $pk \leftarrow \mathbf{T}_{\text{len}}(\mathbf{PK}.\text{seed}, \text{otspkADRS}, \text{tmp})$ 
14 return  $pk$ 

```

ctr_{OTS} (4 bytes)
One node in chain 0 (n bytes)
... ..
One node in chain $\text{len} - 1$ (n bytes)

Figure 1.12: Inhomogeneous WOTS^c signature data format

given by Algorithm 9 generates the inhomogeneous WOTS^c signature. In addition to the n -byte message to be signed, Sign_{OTS} takes $\mathbf{SK}.\text{seed}$ and $\mathbf{PK}.\text{seed}$ from the FlexTree private key and an address as input. The type in the address ADRS must be set to OTS_HASH , and the layer address, tree address, and key pair address must encode the address of the WOTS^c key that is used to sign the message. Firstly, Sign_{OTS} repeatedly hash the n -byte message $\mathbf{n}$ with a counter encoded in an address to obtain the output digest until the conditions

$$\begin{cases} \text{leadingBits}(\text{digest}, z_b) = 0^{z_b} \\ \text{IntSeq}_w(\text{trailingBits}(\text{digest}, 8n - z_b)) \in \mathcal{C}_s(\prod_{j=0}^{l-1} [w_j]) \end{cases} \quad (1.7)$$

are fulfilled (Lines 4 through 8). Then, the least significant $(8n - z_b)$ -bit segment of digest is encoded into a vector $(s_0, \dots, s_{\text{len}-1})$ of len integers in $[w_0] \times \dots \times [w_{\text{len}-1}]$ (Line 10). For each of the len integers, Lines 15 through 18 compute the node in the hash chain that corresponds to the integer. The inhomogeneous WOTS^c signature contains the $n_{\text{ctr}}^{\text{OTS}}$ -byte counter and the len computed n -byte nodes.

Algorithm 9: $\text{Sign}_{\text{OTS}}(\mathbf{n}, \mathbf{SK}.\text{seed}, \mathbf{PK}.\text{seed}, \text{ADRS})$

Input: An n -byte element $\mathbf{n}$ (typically the root of an XMSS tree or PORs tree),
Secret seed $\mathbf{SK}.\text{seed}$, public seed $\mathbf{PK}.\text{seed}$, and the address ADRS

Output: OTS signature

```

1 rootHashAddr  $\leftarrow$  ADRS
2 rootHashAddr.setTypeAndClear(ROOT_HASH)
3 rootHashAddr.setKeyPairAddress(ADRS.getKeyPairAddress())
4 digest  $\leftarrow \mathbf{H}_{\text{Root}}(\mathbf{PK}.\text{seed}, \text{rootHashAddr}, \mathbf{n})$ 
5 cs  $\leftarrow \left\lfloor \frac{(w_0-1)+\dots+(w_{\text{len}-1}-1)}{2} \right\rfloor$ 
6 while leadingBits(digest,  $z_b$ )  $\neq 0^{z_b}$  or
   IntSeqw(trailingBits(digest,  $8n - z_b$ ))  $\notin \mathcal{C}_s(\prod_{j=0}^{l-1} [w_j])$  do
7   rootHashAddr.counterIncrement()
8   digest  $\leftarrow \mathbf{H}_{\text{Root}}(\mathbf{PK}.\text{seed}, \text{rootHashAddr}, \mathbf{n})$ 
9 ctr  $\leftarrow$  rootHashAddr.getCounter()
10 ( $s_0, \dots, s_{\text{len}-1}$ )  $\leftarrow$  IntSeqw(trailingBits(digest,  $8n - z_b$ ))
11 skADRS  $\leftarrow$  ADRS
12 skADRS.setTypeAndClear(OTS_PRF)
13 skADRS.setKeyPairAddress(ADRS.getKeyPairAddress())
14 for for  $i$  from 0 to  $\text{len} - 1$  do
15   skADRS.setChainAddress( $i$ )
16    $sk \leftarrow \text{PRF}(\mathbf{PK}.\text{seed}, \mathbf{SK}.\text{seed}, \text{skADRS})$ 
17   ADRS.setChainAddress( $i$ )
18    $\text{sig}[i] \leftarrow \text{chain}(sk, 0, s_i, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
19 return ctr || sig

```

1.7.3 Computing an OTS Public Key From a Signature

Verifying a WOTS^c signature involves computing a public-key value from a message and signature value. Verification succeeds if the correct public-key value is computed, which is determined by using the computed public-key value along with other information to compute a candidate $\mathbf{PK}.\text{root}$ value and then comparing that value to the known value of $\mathbf{PK}.\text{root}$ from the `FlexTree` public key. This section describes `pkFromSigOTS` (Algorithm 10), a function that computes a candidate WOTS^c public key from a WOTS^c signature and corresponding message.

In addition to an n -byte message M and a $(n_{\text{ctr}}^{\text{OTS}} + \text{len} \cdot n)$ -byte signature $\mathbf{SIG}_{\text{OTS}}$, which is interpreted as a $n_{\text{ctr}}^{\text{OTS}}$ -byte counter concatenated with an array of len n -byte strings, the `pkFromSigOTS` function takes $\mathbf{PK}.\text{seed}$ from the `FlexTree` public key and an address as input. The type of the address ADRS must be set to `OTS_HASH`, and the layer address, tree address, and key pair address must encode the address of the WOTS^c key that was used to sign the message.

Lines 1 and 2 of Algorithm 10 extract the counter and array of len n -byte hash chain nodes. Lines 8 through 10 compute the end nodes for each of the chains using the signature value as the starting point and the message value to determine the number of iterations

that need to be performed to get to the end node. Finally, the computed end nodes are compressed by Line 14.

Algorithm 10: $\text{pkFromSig}_{\text{OTS}}(\mathbf{SIG}_{\text{OTS}}, M, \mathbf{PK.seed}, \text{ADRS})$

Input: OTS signature $\mathbf{SIG}_{\text{OTS}}$, message M , public seed $\mathbf{PK.seed}$, and the address ADRS

Output: The OTS public key derived from $\mathbf{SIG}_{\text{OTS}}$

```

1  $\text{ctr} \leftarrow \mathbf{SIG}_{\text{OTS}}.\text{getCounter}()$ 
2  $\text{sig} \leftarrow \mathbf{SIG}_{\text{OTS}}.\text{getNodeArray}()$ 
3  $\text{rootHashAddr} \leftarrow \text{ADRS}$ 
4  $\text{rootHashAddr}.\text{setTypeAndClear}(\text{ROOT\_HASH})$ 
5  $\text{rootHashAddr}.\text{setKeyPairAddress}(\text{ADRS}.\text{getKeyPairAddress}())$ 
6  $\text{digest} \leftarrow \mathbf{H}_{\text{Root}}(\mathbf{PK.seed}, \text{rootHashAddr}, M)$ 
7  $(s_0, \dots, s_{\text{len}-1}) \leftarrow \text{IntSeq}_w(\text{trailingBits}(\text{digest}, 8n - z_b))$ 
8 for  $i$  from 0 to  $\text{len} - 1$  do
9    $\text{ADRS}.\text{setChainAddress}(i)$ 
10   $\text{tmp}[i] \leftarrow \text{chain}(\text{sig}[i], s_i, w - 1 - s_i, \mathbf{PK.seed}, \text{ADRS})$ 
11  $\text{otspkADRS} \leftarrow \text{ADRS}$ 
12  $\text{otspkADRS}.\text{setTypeAndClear}(\text{OTS\_PK})$ 
13  $\text{otspkADRS}.\text{setKeyPairAddress}(\text{ADRS}.\text{getKeyPairAddress}())$ 
14  $\text{pk}_{\text{OTS}} \leftarrow \mathbf{T}_{\text{len}}(\mathbf{PK.seed}, \text{otspkADRS}, \text{tmp})$ 
15 return  $\text{pk}_{\text{OTS}}$ 

```

1.8 eXtended Merkle Signature Scheme (XMSS)

An XMSS tree in layer i has 2^{h_i} leaves with each one being the public key of an inhomogeneous WOTS^c one-time signature. It allows the leaf nodes to be authenticated using a single n -byte XMSS public key (i.e., the root of the XMSS tree). An XMSS signature is $n_{\text{ctr}}^{\text{OTS}} + \text{len} \cdot n + h_i \cdot n$ bytes in length and consists of an inhomogeneous WOTS^c signature and an authentication path as illustrated in Figure 1.13. The authentication path is an array of nodes from the Merkle tree – one from each level of the tree, except for the root – that allows the verifier to compute the root of the XMSS tree when used in conjunction with the public key of the one-time signature.

$\mathbf{SIG}_{\text{OTS}}$	$(4 + \text{len} \cdot n \text{ bytes})$
The 0-th element of $\mathbf{AUTH}$	$(n \text{ bytes})$
$\dots$	$\dots$
The $(h_i - 1)$ -th element of $\mathbf{AUTH}$	$(n \text{ bytes})$

Figure 1.13: XMSS signature data format for the i -th layer

1.8.1 Generating a Merkle Hash Tree

The `xmssNode` function (Algorithm 11) computes the nodes of an XMSS tree. The `xmssNode` function takes as input `SK.seed` and `PK.seed` from the private key, a target node index i (the index of the node being computed), a target node height z (the height within the Merkle tree of the node being computed) and an address. The address `ADRS` must have the layer address and tree address set to the XMSS tree within which the node is being computed.

Each node in an XMSS tree is the root of a subtree, and Algorithm 11 computes the root of the subtree recursively. If the subtree consists of a single leaf node, then the function simply returns the value of the corresponding WOTS^c public key (Lines 2 through 4). Otherwise, the function computes the roots of the left subtree (Line 6) and right subtree (Line 7) and hashes them together (Line 11).

Algorithm 11: `xmssNode(SK.seed,  $i$ ,  $z$ , PK.seed, ADRS)`

Input: Secret `SK.seed`, target node index i , target node height z , public seed `PK.seed`, address `ADRS`

Output: n -byte XMSS tree node

```

1 if  $z = 0$  then
2   ADRS.setTypeAndClear(OTS_HASH)
3   ADRS.setKeyPairAddress( $i$ )
4    $node \leftarrow \text{pkGen}_{\text{OTS}}(\text{SK.seed}, \text{PK.seed}, \text{ADRS})$ 
5 else
6    $lnode \leftarrow \text{xmssNode}(\text{SK.seed}, 2i, z - 1, \text{PK.seed}, \text{ADRS})$ 
7    $rnode \leftarrow \text{xmssNode}(\text{SK.seed}, 2i + 1, z - 1, \text{PK.seed}, \text{ADRS})$ 
8   ADRS.setTypeAndClear(XMSS_TREE)
9   ADRS.setTreeHeight( $z$ )
10  ADRS.setTreeIndex( $i$ )
11   $node \leftarrow \mathbf{H}(\text{PK.seed}, \text{ADRS}, lnode \parallel rnode)$ 
12 return  $node$ 

```

1.8.2 Generating an XMSS Signature

The `SignXMSS` function (Algorithm 12) creates an XMSS signature on an n -byte message M by creating an authentication path (Lines 3 through 4) and signing M with the appropriate WOTS^c key (Lines 5 through 7). In addition to M , `SignXMSS` takes `SK.seed` and `PK.seed` from the `FlexTree` private key, an address, and an index as input. The address `ADRS` must have the layer address and tree address set to the XMSS key that is being used to sign the message, and the index `idx` must be the index of the WOTS^c key within the XMSS tree that will be used to sign the message. The authentication path consists of the sibling nodes of each node that is on the path from the WOTS^c key used to the root. In Line 3 of Algorithm 12, $\lfloor \text{idx}/2^j \rfloor$ is the on path node, and $\lfloor \text{idx}/2^j \rfloor \oplus 1$ is the authentication path node. Line 4 computes the value of the authentication path node.

1.8.3 Computing an XMSS Public Key from a XMSS Signature

Verifying an XMSS signature from the i -th layer involves computing a public-key value from a message and an XMSS signature value. Verification succeeds if the correct public-key

Algorithm 12: $\text{Sign}_{\text{XMSS}}(M, \mathbf{SK}.\text{seed}, \text{idx}, \mathbf{PK}.\text{seed}, \text{ADRS})$

Input: n -byte message M , secret seed $\mathbf{SK}.\text{seed}$, index idx , public seed $\mathbf{PK}.\text{seed}$, address ADRS

Output: XMSS signature σ_{XMSS}

```

1  $i \leftarrow \text{ADRS}.\text{getLayerAddress}()$ 
2 for  $j$  from 0 to  $h_i - 1$  do
3    $k \leftarrow \lfloor \text{idx}/2^j \rfloor \oplus 1$ 
4    $\text{Auth}[j] \leftarrow \text{xmssNode}(\mathbf{SK}.\text{seed}, k, j, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
5  $\text{ADRS}.\text{setTypeAndClear}(\text{OTS\_HASH})$ 
6  $\text{ADRS}.\text{setKeyPairAddress}(\text{idx})$ 
7  $\sigma_{\text{OTS}} \leftarrow \text{Sign}_{\text{OTS}}(M, \mathbf{SK}.\text{seed}, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
8  $\sigma_{\text{XMSS}} \leftarrow \sigma_{\text{OTS}} \parallel \text{Auth}$ 
9 return  $\sigma_{\text{XMSS}}$ 

```

value is computed, which is determined by using the computed public-key value along with other information to compute a candidate $\mathbf{PK}.\text{root}$ value and then comparing that value to the known value of $\mathbf{PK}.\text{root}$ from the **FlexTree** public key. This section describes $\text{pkFromSig}_{\text{XMSS}}$ (Algorithm 13), a function that computes a candidate XMSS public key from an XMSS signature and corresponding message.

In addition to an n -byte message M and an $(n_{\text{ctr}}^{\text{OTS}} + (\text{len} + h_i) \cdot n)$ -byte signature $\mathbf{SIG}_{\text{XMSS}}$, $\text{pkFromSig}_{\text{XMSS}}$ takes $\mathbf{PK}.\text{seed}$ from the **FlexTree** public key, an address, and an index as input. The address ADRS must be set to the layer address and tree address of the XMSS key that was used to sign the message, and the index idx must be the index of the WOTS^c key within the XMSS tree that was used to sign the message. Algorithm 13 begins by computing the WOTS^c public key in Lines 1 through 5. The root is then computed in Lines 6 through 17. Starting with the leaf node (i.e., the WOTS^c public key), a node at each level of the tree is computed by hashing together the node computed in the previous iteration with the corresponding authentication path node. In Lines 13 and 16, **AUTH** is interpreted as an array of h_i n -byte strings.

Algorithm 13: $\text{pkFromSig}_{\text{XMSS}}(\text{idx}, \text{SIG}_{\text{XMSS}}, M, \text{PK.seed}, \text{ADRS})$

Input: Index idxLeaf , XMSS signature SIG_{XMSS} , n -byte message M , public seed PK.seed , and address ADRS

Output: n -byte root value of an XMSS tree

```

1  ADRS.setTypeAndClear(OTS_HASH)
2  ADRS.setKeyPairAddress(idx)
3  SIG  $\leftarrow$  SIGXMSS.getSigOTS()
4  AUTH  $\leftarrow$  SIG.getAuthXMSS()
5  node[0]  $\leftarrow$  pkFromSigOTS(SIG, M, PK.seed, ADRS)
6  ADRS.setTypeAndClear(XMSS_TREE)
7  ADRS.setTreeIndex(idx)
8   $i \leftarrow$  ADRS.getLayerAddress()
9  for  $k$  from 0 to  $h_i - 1$  do
10 |   ADRS.setTreeHeight( $k + 1$ )
11 |   if  $\lfloor \text{idx}/2^k \rfloor$  is even then
12 |       ADRS.setTreeIndex(ADRS.getTreeIndex()/2)
13 |       node[1]  $\leftarrow$  H(PK.seed, ADRS, node[0] || AUTH[k])
14 |   else
15 |       ADRS.setTreeIndex((ADRS.getTreeIndex() - 1)/2)
16 |       node[1]  $\leftarrow$  H(PK.seed, ADRS, AUTH[k] || node[0])
17 |   node[0]  $\leftarrow$  node[1]
18 return node[0]
```

1.9 The FlexTree Hypertree Signature

As described in Section 1.4, a **FlexTree** hypertree is a tree of **XMSS** trees. The **XMSS** keys at the lowest layer are used to sign **PORS** public keys, and the **XMSS** keys at every other layer are used to sign the **XMSS** public keys at the layer below. The top layer (layer $d - 1$) is a single **XMSS** tree, and the public key of this **XMSS** key pair (i.e., the root of the Merkle tree) is the public key of the hypertree (**PK.root**). The lowest layer has 2^{h-h_0} **XMSS** trees, providing 2^h **WOTS^C** keys to sign **PORS** keys.

1.9.1 Hypertree Signature Generation

A **FlexTree** hypertree signature is $d \cdot n_{\text{ctr}}^{\text{OTS}} + (d \cdot \text{len} + h) \cdot n$ bytes in length and consists of a sequence of d **XMSS** signatures, starting with a signature generated using an **XMSS** key at the lowest layer and ending with a signature generated using the **XMSS** key at the top layer (see Figure 1.14).

In addition to the n -byte message M , the sign_{HT} function given by Algorithm 14 takes as input **SK.seed** and **PK.seed** from the **FlexTree** private key, the index **idxTree** of the **XMSS** tree at the lowest layer that will sign the message, and the index **idxLeaf** of the **WOTS^C** key within the **XMSS** tree that will sign the message .

SIG_{XMSS} for layer 0 $(4 + (h_0 + \text{len}) \cdot n \text{ bytes})$
SIG_{XMSS} for layer 1 $(4 + (h_1 + \text{len}) \cdot n \text{ bytes})$
...
SIG_{XMSS} for layer $d - 1$ $(4 + (h_{d-1} + \text{len}) \cdot n \text{ bytes})$

Figure 1.14: Hypertree signature data format

Algorithm 14 begins in Lines 1 through 3 by signing M with the specified **XMSS** key using the **WOTS^C** key within that **XMSS** key specified by **idxLeaf**. The **XMSS** public key is obtained (Line 5 or 14) for each successive layer and signed by the appropriate key at the next higher level (Lines 7 through 11).

1.9.2 Hypertree Signature Verification

Hypertree signature verification works by making d calls to **pkFromSig_{XMSS}** (Algorithm 13) and comparing the result to the public key of the hypertree.

In addition to the n -byte message M and the $(d \cdot n_{\text{ctr}}^{\text{OTS}} + (d \cdot \text{len} + h) \cdot n)$ -byte signature **SIG_{HT}**, **verify_{HT}** (Algorithm 15) takes as input **PK.seed** and **PK.root** from the **FlexTree** public key, the index **idxTree** of the **XMSS** tree at the lowest layer that signed the message, and the index **idxLeaf** of the **WOTS^C** key within the **XMSS** tree that signed the message.

At each layer, either the message M or the computed public key of the **XMSS** key at the lower layer is provided along with the appropriate **XMSS** signature to **pkFromSig_{XMSS}** in order to obtain the layer's computed **XMSS** public key. If the computed **XMSS** public key of the top layer tree is the same as the known hypertree public key, **PK.root**, then verification succeeds.

Algorithm 14: $\text{sign}_{\text{HT}}(M, \mathbf{SK}.\text{seed}, \mathbf{PK}.\text{seed}, \text{idxTree}, \text{idxLeaf})$

Input: Message $M \in \mathbb{F}_2^n$, private seed $\mathbf{SK}.\text{seed}$, public seed $\mathbf{PK}.\text{seed}$, XMSS tree index idxTree , leaf index idxLeaf

Output: Hypertree signature σ_{HT}

```

1  ADRS  $\leftarrow$  toByte(0, 32)
2  ADRS.setTreeAddress(idxTree)
3   $\sigma_{\text{XMSS}} \leftarrow \text{Sign}_{\text{XMSS}}(M, \mathbf{SK}.\text{seed}, \text{idxLeaf}, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
4   $\sigma_{\text{HT}} \leftarrow \sigma_{\text{XMSS}}$ 
5   $root \leftarrow \text{pkFromSig}_{\text{XMSS}}(\text{idxLeaf}, \sigma_{\text{XMSS}}, M, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
6  for  $j$  from 1 to  $d - 1$  do
7       $\text{idxLeaf} \leftarrow \text{idxTree} \bmod 2^{h_j}$ 
8       $\text{idxTree} \leftarrow \text{idxTree} \gg h_j$ 
9      ADRS.setLayerAddress( $j$ )
10     ADRS.setTreeAddress(idxTree)
11      $\sigma_{\text{XMSS}} \leftarrow \text{Sign}_{\text{XMSS}}(root, \mathbf{SK}.\text{seed}, \text{idxLeaf}, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
12      $\sigma_{\text{HT}} \leftarrow \sigma_{\text{HT}} \parallel \sigma_{\text{XMSS}}$ 
13     if  $j < d - 1$  then
14          $root \leftarrow \text{pkFromSig}_{\text{XMSS}}(\text{idxLeaf}, \sigma_{\text{XMSS}}, root, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
15 return  $\sigma_{\text{HT}}$ 

```

1.10 The PORS+FP Few-time Signature

PORS+FP [4] is a few-time signature scheme that is used to sign the digests of the actual messages. Unlike one-time signatures, for which security is no longer guaranteed if a key is used more than once, the security of a PORS+FP key degrades gradually as the number of signatures increases. PORS+FP uses four parameters: t , k , m_{MAX} , and $n_{\text{ctr}}^{\text{FTS}}$. A PORS+FP private key consists of a set of t n -byte strings, all of which are pseudorandomly generated from the secret seed $\mathbf{SK}.\text{seed}$. The set of secret values is formed into a Merkle tree called PORS tree, and the root of the tree is the PORS+FP public key. As shown in Figure 1.15, a PORS+FP signature on an n -byte message digest consists of an $n_{\text{ctr}}^{\text{FTS}}$ -byte counter, k elements from the PORS+FP private key, along with m_{MAX} authentication nodes for the private key elements. Note that when the number m' of the authentication nodes returned by the octopus algorithm is less than m_{MAX} , we pad the signature with $(m_{\text{MAX}} - m') \cdot n$ zero bytes.

1.10.1 Generating PORS+FP Secret Values

The $\text{skGen}_{\text{PORS}}$ function (Algorithm 16) generates the t n -byte strings of the PORS+FP private key. The function takes $\mathbf{SK}.\text{seed}$ and $\mathbf{PK}.\text{seed}$ from the FlexTree private key, an address, and an index as input. The type in the address ADRS must be set to PORS_PRF, and the tree address and key pair address must be set to the index of the WOTS^c key within the XMSS tree that signs the PORS+FP key. The layer address must be set to zero. The index j is the index of the PORS+FP secret value.

Algorithm 15: $\text{verify}_{\text{HT}}(M, \text{SIG}_{\text{HT}}, \text{PK.seed}, \text{idxTree}, \text{idxLeaf}, \text{PK.root})$

Input: Message $M \in \mathbb{F}_2^n$, signature SIG_{HT} , public seed PK.seed , XMSS tree index idxTree , leaf index idxLeaf , public key PK.root

Output: Boolean

```

1  ADRS  $\leftarrow$  toByte(0, 32)
2  ADRS.setTreeAddress(idxTree)
3   $\text{SIG}_{\text{tmp}} \leftarrow \text{SIG}_{\text{HT}}.\text{getXMSSSignature}(0)$ 
4   $\text{ctr}_{\text{OTS}} \leftarrow \text{SIG}_{\text{tmp}}.\text{getCounter}_{\text{OTS}}()$ 
5  if  $\text{ctr}_{\text{OTS}}$  is valid then
6     $\text{node} \leftarrow \text{pkFromSig}_{\text{XMSS}}(\text{idxLeaf}, \text{SIG}_{\text{tmp}}, M, \text{PK.seed}, \text{ADRS})$ 
7    for  $j$  from 1 to  $d - 1$  do
8       $\text{idxLeaf} \leftarrow \text{idxTree} \bmod 2^{h_j}$ 
9       $\text{idxTree} \leftarrow \text{idxTree} \gg h_j$ 
10     ADRS.setLayerAddress( $j$ )
11     ADRS.setTreeAddress(idxTree)
12      $\text{ctr}_{\text{OTS}} \leftarrow \text{SIG}_{\text{tmp}}.\text{getCounter}_{\text{OTS}}()$ 
13     if  $\text{ctr}_{\text{OTS}}$  is valid then
14        $\text{SIG}_{\text{tmp}} \leftarrow \text{SIG}_{\text{HT}}.\text{getXMSSSignature}(j)$ 
15        $\text{node} \leftarrow \text{pkFromSig}_{\text{XMSS}}(\text{idxLeaf}, \text{SIG}_{\text{tmp}}, \text{node}, \text{PK.seed}, \text{ADRS})$ 
16     else
17       return false
18   if  $\text{node} = \text{PK.root}$  then
19     return true
20   else
21     return false
22 else
23   return false

```

Counter ctr ($n_{\text{ctr}}^{\text{FTS}}$ bytes)
Private key element 0 (n bytes)
...
Private key element $k - 1$ (n bytes)
Authentication node 0 (n bytes)
...
Authentication node $m_{\text{MAX}} - 1$ (n bytes)

Figure 1.15: PORS+FP signature data format

1.10.2 Computing the Nodes of a PORS Tree

The $\text{node}_{\text{PORS}}$ function (Algorithm 17) computes the nodes of a PORS tree. It is the same as xmssNode , except that the leaf nodes are the hashes of the PORS+FP secret values instead

Algorithm 16: $\text{skGen}_{\text{PORS}}(\mathbf{SK}.\text{seed}, \mathbf{PK}.\text{seed}, \text{ADRS}, j)$

Input: Secret seed $\mathbf{SK}.\text{seed}$, public seed $\mathbf{PK}.\text{seed}$, address ADRS , and an index j

Output: The j th n -byte secret element of the secret key of PORS+FP

```

1 skADRS  $\leftarrow$  ADRS
2 skADRS.setTypeAndClear(PORS_PRF)
3 skADRS.setKeyPairAddress(ADRS.getKeyPairAddress())
4 skADRS.setTreeIndex( $j$ )
5 return PRF( $\mathbf{PK}.\text{seed}, \mathbf{SK}.\text{seed}, \text{skADRS}$ )

```

of WOTS^C public keys. The $\text{node}_{\text{PORS}}$ function takes as input $\mathbf{SK}.\text{seed}$ and $\mathbf{PK}.\text{seed}$ from the FlexTree private key, a target node index i (the index of the node being computed), a target node height z (the height within the PORS tree of the node being computed), and an address. The address ADRS must have the layer address set to zero (since the XMSS tree that signs a PORS+FP key is always at layer 0), the tree address set to the XMSS tree that signs the PORS+FP key, the type set to FTS_TREE , and the key pair address set to the index of the WOTS^C key within the XMSS tree that signs the PORS+FP key. The target node height and index must satisfy $z \leq h_{\text{PORS}}$, $i < 2s$ for $z = 0$ and $i < 2^{h_{\text{PORS}} - z}$ for $z > 0$.

Each node in the PORS tree is the root of a subtree, and Algorithm 17 computes the root of a subtree recursively. If the subtree consists of a single leaf node, then the function simply returns a hash of the node's private n -byte string (Lines 2 through 11). Otherwise, the function computes the roots of the left subtree (Line 13) and right subtree (Line 14) and hashes them together (Line 17).

1.10.3 Generating a PORS+FP Signature

The sign_{FTS} function (Algorithm 19) signs an n -byte message digest md . In addition to the message digest, sign_{FTS} takes $\mathbf{SK}.\text{seed}$ and $\mathbf{PK}.\text{seed}$ from the FlexTree private key and an address as input. The address ADRS must have the layer address set to zero (since the XMSS tree that signs a PORS+FP key is always at layer 0), the tree address set to the XMSS tree that signs the PORS+FP key, the type set to FTS_TREE , and the key pair address set to the index of the WOTS^C key within the XMSS tree that signs the PORS+FP key.

The $\text{sign}_{\text{PORS}}$ function relies on octopus (Algorithm 18), which computes the indices of the minimal set of authentication nodes for k leaves of a PORS tree.

The $\text{sign}_{\text{PORS}}$ function repeatedly hashes the message digest md with a counter and encodes the result as an element $\mathcal{I} \in \binom{[t]}{k}$ until the size of the authentication set returned by the octopus algorithm for the PORS tree leaves indexed by $\mathcal{I}$ is not greater than m_{MAX} . Each of the integers in $\mathcal{I}$ is used to select a secret value from the t secret values of PORS+FP (Line 9 and Line 10 of Algorithm 19). For the k secret values selected, the authentication nodes are computed and added to the signature (Lines 11 through 12 of Algorithm 19).

1.10.4 Computing a PORS+FP Public Key from a Signature

Verifying a PORS+FP signature involves computing a public-key value from a message digest and a PORS+FP signature value. Verification succeeds if the correct public-key value is computed, which is determined by verifying the hypertree signature on the computed public-key value using the FlexTree public key. This section describes $\text{pkFromSig}_{\text{PORS}}$ (Algorithm 20),

Algorithm 17: $\text{node}_{\text{PORS}}(\mathbf{SK}.\text{seed}, i, z, \mathbf{PK}.\text{seed}, \text{ADRS})$

Input: Secret $\mathbf{SK}.\text{seed}$, target node index i , target node height z , public seed $\mathbf{PK}.\text{seed}$, address ADRS
Output: An n -byte PORS tree node

```

1 ASSERT  $\text{ADRS}.\text{addrType} = \text{FTS\_TREE}$ 
2 if  $z = 0$  then
3    $sk \leftarrow \text{skGen}_{\text{PORS}}(\mathbf{SK}.\text{seed}, \mathbf{PK}.\text{seed}, \text{ADRS}, i)$ 
4    $\text{ADRS}.\text{setTreeHeight}(0)$ 
5    $\text{ADRS}.\text{setTreeIndex}(i)$ 
6    $\text{node} \leftarrow \mathbf{F}(\mathbf{PK}.\text{seed}, \text{ADRS}, sk)$ 
7 else if  $z = 1$  and  $i > s - 1$  then
8    $sk \leftarrow \text{skGen}_{\text{PORS}}(\mathbf{SK}.\text{seed}, \mathbf{PK}.\text{seed}, \text{ADRS}, i + s)$ 
9    $\text{ADRS}.\text{setTreeHeight}(0)$ 
10   $\text{ADRS}.\text{setTreeIndex}(i)$ 
11   $\text{node} \leftarrow \mathbf{F}(\mathbf{PK}.\text{seed}, \text{ADRS}, sk)$ 
12 else
13   $l\text{node} \leftarrow \text{node}_{\text{PORS}}(\mathbf{SK}.\text{seed}, 2i, z - 1, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
14   $r\text{node} \leftarrow \text{node}_{\text{PORS}}(\mathbf{SK}.\text{seed}, 2i + 1, z - 1, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
15   $\text{ADRS}.\text{setTreeHeight}(z)$ 
16   $\text{ADRS}.\text{setTreeIndex}(i)$ 
17   $\text{node} \leftarrow \mathbf{H}(\mathbf{PK}.\text{seed}, \text{ADRS}, l\text{node} \parallel r\text{node})$ 
18 return  $\text{node}$ 

```

Algorithm 18: $\text{octopus}(\mathcal{I})$

Input: The index set $\mathcal{I} \subseteq \{0, \dots, t - 1\}$ with $|\mathcal{I}| = k$
Output: The node set $\mathbb{A}$, where the j -th node $\mathbf{n}$ with height i is represented as a pair (i, j) , and the height i can be accessed as $\mathbf{n}.\text{height}$ and j can be accessed as $\mathbf{n}.\text{idx}$

```

1  $\mathbb{I} \leftarrow \{i \in \mathcal{I} : i < 2s\}$ 
2  $\mathbb{P} \leftarrow \{i - s : i \in \mathcal{I}, i \geq 2s\}$ 
3  $\mathbb{A} \leftarrow \emptyset$ 
4 for  $i$  from 0 to  $h_{\text{PORS}} - 1$  do
5   for  $j \in \mathbb{I}$  do
6      $\mathbb{P} \leftarrow \mathbb{P} \cup \{\text{par}(j)\}$ 
7     if  $\text{sib}(j) \notin \mathbb{I}$  then
8        $\mathbb{A} \leftarrow \mathbb{A} \cup \{(i, \text{sib}(j))\}$ 
9    $\mathbb{I} \leftarrow \mathbb{P}$ 
10   $\mathbb{P} \leftarrow \emptyset$ 
11 return  $\mathbb{A}$ 

```

a function that computes a candidate PORS+FP public key from a PORS+FP signature and corresponding message digest.

Algorithm 19: $\text{sign}_{\text{FTS}}(\text{md}, \text{SK.seed}, \text{PK.seed}, \text{ADRS}, \mathbf{R})$

Input: n -byte md , secret seed SK.seed , public seed PK.seed , and address ADRS

Output: The PORS+FP Signature of md

```

1   $\text{ctr} \leftarrow -1$ 
2   $\sigma_{\text{PORS}} \leftarrow \emptyset$ 
3  repeat
4     $\text{ctr} \leftarrow \text{ctr} + 1$ 
5     $\mathcal{I} \leftarrow \rho_{\binom{t}{k}}^{(\binom{t}{k})} (\mathbf{H}_{\text{PORS}}(\mathbf{R}, \text{ctr}, \text{md}) \bmod \binom{t}{k})$ 
6     $\mathbb{A} \leftarrow \text{octopus}(\mathcal{I})$ 
7  until  $|\mathbb{A}| \leq m_{\text{MAX}}$  ;
8   $\sigma_{\text{PORS}} \leftarrow \sigma_{\text{PORS}} \parallel \text{ctr}$ 
9  for  $i \in \mathcal{I}$  do
10    $\sigma_{\text{PORS}} \leftarrow \sigma_{\text{PORS}} \parallel \text{skGen}_{\text{PORS}}(\text{SK.seed}, \text{PK.seed}, \text{ADRS}, i)$ 
11 for  $i$  from 0 to  $|\mathbb{A}| - 1$  do
12    $\sigma_{\text{PORS}} \leftarrow \sigma_{\text{PORS}} \parallel \text{node}_{\text{PORS}}(\text{SK.seed}, \mathbb{A}[i].\text{idx}, \mathbb{A}[i].\text{height}, \text{PK.seed}, \text{ADRS})$ 
13 return  $\sigma_{\text{PORS}}$ 

```

In addition to an n -byte message digest md and an $(n_{\text{ctr}}^{\text{FTS}} + k \cdot n + m_{\text{MAX}} \cdot n)$ -byte PORS+FP signature SIG_{PORS} , $\text{pkFromSig}_{\text{PORS}}$ takes PK.seed from the **FlexTree** public key and an address as input. The $\text{pkFromSig}_{\text{PORS}}$ function begins by extracting the counter ctr and σ'_{PORS} , the concatenation of k n -byte element and their authentication nodes. Then $\text{pkFromSig}_{\text{PORS}}$ calls the function computeRoot given by Algorithm 21 to compute the root of the corresponding PORS tree, which is the public key. In Algorithm 21 the sets **Current** and **Parentss** maintain some nodes in a specific level of a PORS tree, where a node is represented as a tuple $\mathbf{n} = (i, v)$, and i can be accessed as $\mathbf{n.idx}$ denoting the index of the node within the specific level and v can be accessed as $\mathbf{n.val}$ denoting the value of the node.

Algorithm 20: $\text{pkFromSig}_{\text{PORS}}(\text{SIG}_{\text{PORS}}, \text{md}, \text{PK.seed}, \text{ADRS}, \mathbf{R})$

Input: PORS signature σ_{PORS} , n -byte md , public seed PK.seed , and address ADRS

Output: PORS public key pk_{PORS}

```

1   $\text{ctr}, \sigma'_{\text{PORS}} \xleftarrow{\text{parse}} \sigma_{\text{PORS}}$ 
2   $\mathcal{I} \leftarrow \rho_{\binom{t}{k}}^{(\binom{t}{k})} (\mathbf{H}_{\text{PORS}}(\mathbf{R}, \text{ctr}, \text{md}) \bmod \binom{t}{k})$ 
3   $\text{root} \leftarrow \text{computeRoot}(\sigma'_{\text{PORS}}, \mathcal{I}, \text{PK.seed}, \text{ADRS})$ 
4  return  $\text{root}$ 

```

1.11 The FlexTree Signature Scheme

FlexTree uses the hypertree and the PORS+FP keys to create a stateless hash-based signature scheme. The **FlexTree** private key contains a secret seed value and a secret PRF key. The public key consists of a key identifier PK.seed and the root of the hypertree. A signature is created by hashing the message, using part of the message digest to select a PORS+FP key,

Algorithm 21: computeRoot($\sigma'_{\text{PORS}}, \mathcal{I}, \text{PK.seed}, \text{ADRS}$)

Input: The remaining part σ'_{PORS} of a PORS signature after removing ctr, the set $\mathcal{I}$ of selected PORS leaves (a k -element subset of $\{0, \dots, t-1\}$), public seed **PK.seed**, and address **ADRS**.

Output: The root of the PORS tree.

```

1   $s \leftarrow t - 2^{h_{\text{PORS}} - 1}$ 
2   $\mathcal{I}^{(0)} \leftarrow \{i \in \mathcal{I} : i < 2s\}$ 
3   $\mathcal{I}^{(1)} \leftarrow \{i : i - s \in \mathcal{I}, i \geq 2s\}$ 
4   $\{sk_i\}_{i \in \mathcal{I}}, \mathcal{A} \xleftarrow{\text{parse}} \sigma'_{\text{PORS}}$ 
5  Current  $\leftarrow \emptyset$ 
6  Parents  $\leftarrow \emptyset$ 
7  for  $i \in \mathcal{I}^{(0)}$  do
8    ADRS.setTreeHeight(0)
9    ADRS.setTreeIndex( $i$ )
10   Current  $\leftarrow \text{Current} \cup \{(i, \mathbf{F}(\text{PK.seed}, \text{ADRS}, sk_i))\}$ 
11   $j^\dagger \leftarrow 0$ ; //  $j^\dagger$  keeps track of the current position in  $\mathcal{A}$ 
12  for  $z$  from 0 to  $h_{\text{PORS}} - 1$  do
13     $j \leftarrow 0$ 
14    while  $j < |\text{Current}|$  do
15      if  $j + 1 < |\text{Current}|$  and  $\text{Current}[j+1].\text{idx} \oplus \text{Current}[j].\text{idx} = 1$  then
16         $\text{temp} \leftarrow \text{Current}[j].\text{val} \parallel \text{Current}[j+1].\text{val}$ 
17         $j \leftarrow j + 2$ 
18      else
19        if  $\text{Current}[j].\text{idx} \wedge 1 = 0$  then
20           $\text{temp} \leftarrow \text{Current}[j].\text{val} \parallel \mathcal{A}[j^\dagger]$ 
21        else
22           $\text{temp} \leftarrow \mathcal{A}[j^\dagger] \parallel \text{Current}[j].\text{val}$ 
23           $j^\dagger \leftarrow j^\dagger + 1$ 
24           $j \leftarrow j + 1$ 
25      ADRS.setTreeHeight( $z + 1$ )
26      ADRS.setTreeIndex( $\text{Current}[j].\text{idx} \gg 1$ )
27      Parents  $\leftarrow \text{Parents} \cup \{(\text{Current}[j].\text{idx} \gg 1, \mathbf{H}(\text{PK.seed}, \text{ADRS}, \text{temp}))\}$ 
28    if  $z = 0$  then
29      Current  $\leftarrow \text{Parents}$ 
30      for  $i \in \mathcal{I}^{(1)}$  do
31        ADRS.setTreeHeight(0)
32        ADRS.setTreeIndex( $i$ )
33        Current  $\leftarrow \text{Current} \cup \{(i, \mathbf{F}(\text{PK.seed}, \text{ADRS}, sk_{i+s}))\}$ 
34  return Current[0].val

```

signing other bits from the message digest with the PORS+FP key, and generating a hypertree signature for the PORS+FP key. The parameters for **FlexTree** are those specified previously for WOTS^c, XMSS, the **FlexTree** hypertree, and PORS+FP. **FlexTree** uses one additional

parameter m , which is the length in bytes of the message digest. It is computed as

$$m = \left\lceil \frac{h - h_0}{8} \right\rceil + \left\lceil \frac{h_0}{8} \right\rceil + n.$$

FlexTree uses h bits of the message digest to select a **PORS+FP** key: $h - h_0$ bits to select an **XMSS** tree at the lowest layer and h_0 bits to select a **WOTS^C** key and corresponding **PORS+FP** key from that tree. n bytes of the digest are signed by the selected **PORS+FP** key. While only $h + 8n$ bits of the message digest are used, implementation is simplified by extracting the necessary bits from a slightly larger digest. This section describes the functions for **FlexTree** key generation, signature generation, and signature verification. In the functions in this section, where randomness is required, the random values are provided as inputs to the functions.

1.11.1 FlexTree Key Generation

FlexTree public keys contain two elements. The first is an n -byte public seed **PK.seed**, which is used in many hash function calls to provide domain separation between different **FlexTree** key pairs. The second value is the hypertree public key **PK.root** (i.e., the root of the top layer **XMSS** tree). **PK.seed** shall be generated by a random bit generator supporting at least $8n$ bits of security strength. The **FlexTree** private key contains two random, secret n -byte values, including **SK.seed** and **SK.prf**. **SK.seed** is used to generate all of the OTS and FTS key elements. **SK.prf** is used to generate a randomization value for the randomized hashing of the message in **FlexTree**. The private key also includes a copy of the public key. Both **SK.seed** and **SK.prf** shall be generated by a random bit generator supporting at least $8n$ bits of security strength. Algorithm 22 generates a **FlexTree** key pair. Lines 1 through 3 compute the root of the top layer **XMSS** tree. Line 4 bundles the three inputs and the computed **PK.root** into the private and public keys.

Algorithm 22: $\text{keyGen}_{\text{SLH}}(\text{SK.seed}, \text{SK.prf}, \text{PK.seed})$

Input: Secret seed **SK.seed**, PRF key **SK.prf**, public seed **PK.seed**

Output: **FlexTree** key pair (**SK**, **PK**)

```

1 ADRS  $\leftarrow$  toByte(0, 32)
2 ADRS.setLayerAddress( $d - 1$ )
3 PK.root  $\leftarrow$  xmssNode(SK.seed, 0,  $h_{d-1}$ , PK.seed, ADRS)
4 return ((SK.seed, SK.prf, PK.seed, PK.root), (PK.seed, PK.root))
```

1.11.2 FlexTree Signature Generation

A **FlexTree** signature consists of an n -byte randomization string, a **PORS+FP** signature, and a hypertree signature. Generating a **FlexTree** signature (Algorithm 23) begins by creating an m -byte message digest (Lines 2 through 4). A PRF is used to create a message randomizer (Line 3), and it is hashed along with the message to create the digest (Line 4). Bits are then extracted from the message digest to be signed by the **PORS+FP** key (Line 9), to select an **XMSS** tree (Lines 5 and 7), and to select a **WOTS^C** key and corresponding **PORS+FP** key within that **XMSS** tree (Lines 6 and 8). Next, the **PORS+FP** signature is computed (Lines 10 through

R (n bytes)
SIG_{FTS} ($4 + k \cdot n + m_{\text{MAX}} \cdot n$ bytes)
SIG_{HT} ($4 \cdot d + (d \cdot \text{len} + h) \cdot n$ bytes)

Figure 1.16: FlexTree signature data format

Algorithm 23: $\text{signInternal}_{\text{SLH}}(\text{msg}, \text{SK}, \text{addrnd})$

Input: Message msg , secret key $\text{SK} = (\text{SK.seed}, \text{SK.prf}, \text{PK.seed}, \text{PK.root})$, and an n -byte random number addrnd (optional)
Output: FlexTree signature σ

```

1  $\text{ADRS} \leftarrow \text{toByte}(0, 32)$ 
2  $\text{optRand} \leftarrow \text{addrnd}$ 
3  $\mathbf{R} \leftarrow \text{PRF}_{\text{MSG}}(\text{SK.prf}, \text{optRand}, \text{msg})$ 
4  $\text{digest} \leftarrow \mathbf{H}_{\text{MSG}}(\mathbf{R}, \text{PK.seed}, \text{PK.root}, \text{msg})$ 
5  $\text{idxTree} \leftarrow \text{digest}[0 : \lceil (h - h_0)/8 \rceil]$ 
6  $\text{idxLeaf} \leftarrow \text{digest}[\lceil (h - h_0)/8 \rceil : \lceil (h - h_0)/8 \rceil + \lceil h_0/8 \rceil]$ 
7  $\text{idxTree} \leftarrow \text{toInt}(\text{idxTree}, \lceil (h - h_0)/8 \rceil) \bmod 2^{h-h_0}$ 
8  $\text{idxLeaf} \leftarrow \text{toInt}(\text{idxLeaf}, \lceil h_0/8 \rceil) \bmod 2^{h_0}$ 
9  $\text{md} \leftarrow \text{digest}[\lceil (h - h_0)/8 \rceil + \lceil h_0/8 \rceil : \lceil (h - h_0)/8 \rceil + \lceil h_0/8 \rceil + n]$ 
10  $\text{ADRS.setTreeAddress}(\text{idxTree})$ 
11  $\text{ADRS.setTypeAndClear}(\text{FTS\_TREE})$ 
12  $\text{ADRS.setKeyPairAddress}(\text{idxLeaf})$ 
13  $\sigma_{\text{PORS}} \leftarrow \text{sign}_{\text{PORS}}(\text{md}, \text{SK.seed}, \text{PK.seed}, \text{ADRS}, \mathbf{R})$ 
14  $pk_{\text{PORS}} \leftarrow \text{pkFromSig}_{\text{PORS}}(\sigma_{\text{PORS}}, \text{md}, \text{PK.seed}, \text{ADRS}, \mathbf{R})$ 
15  $\sigma_{\text{HT}} \leftarrow \text{sign}_{\text{HT}}(pk_{\text{PORS}}, \text{SK.seed}, \text{PK.seed}, \text{idxTree}, \text{idxLeaf})$ 
16  $\sigma \leftarrow \mathbf{R} \parallel \sigma_{\text{PORS}} \parallel \sigma_{\text{HT}}$ 
17 return  $\sigma$ 
```

13), and the corresponding PORS+FP public key is obtained (Line 14). Finally, the PORS+FP public key is signed (Line 15).

The message randomizer may be set in either a non-deterministic or deterministic way, depending on whether addrnd is provided as an input. For the “hedged” variant, addrnd is provided as an input, and optRand is set to addrnd . The hedged variant is the default and should be used on platforms where side-channel attacks are a concern. When using the hedged version, addrnd shall be an n -byte random value. While addrnd should ideally be generated by a random bit generator. For the deterministic variant, addrnd is not provided as an input, and optRand is set to PK.seed , which results in signing being deterministic (i.e., signing the same message twice will result in the same signature). The deterministic variant is available for platforms where a random bit generator is not available.

1.11.3 Signature Verification

As with signature generation, **FlexTree** signature verification (Algorithm 24) begins by computing a message digest (Line 9) and then extracting **md** (Line 14), **idxTree** (Lines 10 and 12), and **idxLeaf** (Lines 11 and 13) from the digest. A candidate **PORS+FP** public key is then computed (Line 18), and the signature on the **PORS+FP** key is verified (Line 19). If this signature verification succeeds, then the correct **PORS+FP** public key was computed, and the signature **SIG** on message **msg** is valid.

Algorithm 24: $\text{verifyInternal}_{\text{SLH}}(\text{msg}, \text{SIG}, \text{PK})$

Input: Message **msg**, signature **SIG**, public key **PK** = (**PK.seed**, **PK.root**)
Output: Boolean

```

1 if  $|\text{SIG}| \neq n + n_{\text{ctr}}^{\text{FTS}} + k \cdot n + m_{\text{MAX}} \cdot n + d \cdot (n_{\text{ctr}}^{\text{OTS}} + \text{len} \cdot n) + h \cdot n$  then
2   return false
3 ADRS  $\leftarrow \text{toByte}(0, 32)$ 
4 R  $\leftarrow \text{SIG.getR}()$ 
5 SIGPORS  $\leftarrow \text{SIG.getSIG}_{\text{PORS}}()$ 
6 ctrFTS  $\leftarrow \text{SIG}_{\text{PORS}}.\text{getCounter}_{\text{FTS}}()$ 
7 if ctrFTS is valid then
8   SIGHT  $\leftarrow \text{SIG.getSIG}_{\text{HT}}()$ 
9   digest  $\leftarrow \text{H}_{\text{MSG}}(\text{R}, \text{PK.seed}, \text{PK.root}, \text{msg})$ 
10  idxTree  $\leftarrow \text{digest}[0 : \lceil (h - h_0)/8 \rceil]$ 
11  idxLeaf  $\leftarrow \text{digest}[\lceil (h - h_0)/8 \rceil : \lceil (h - h_0)/8 \rceil + \lceil h_0/8 \rceil]$ 
12  idxTree  $\leftarrow \text{toInt}(\text{idxTree}, \lceil (h - h_0)/8 \rceil \bmod 2^{h-h'})$ 
13  idxLeaf  $\leftarrow \text{toInt}(\text{idxLeaf}, \lceil h_0/8 \rceil \bmod 2^{h_0})$ 
14  md  $\leftarrow \text{digest}[\lceil (h - h_0)/8 \rceil + \lceil h_0/8 \rceil : \lceil (h - h_0)/8 \rceil + \lceil h_0/8 \rceil + n]$ 
15  ADRS.setTreeAddress(idxTree)
16  ADRS.setTypeAndClear(PORS_TREE)
17  ADRS.setKeyPairAddress(idxLeaf)
18   $pk_{\text{PORS}} \leftarrow \text{pkFromSig}_{\text{PORS}}(\text{SIG}_{\text{PORS}}, \text{md}, \text{PK.seed}, \text{ADRS}, \text{R})$ 
19  return  $\text{verify}_{\text{HT}}(pk_{\text{PORS}}, \text{SIG}_{\text{HT}}, \text{PK.seed}, \text{idxTree}, \text{idxLeaf}, \text{PK.root})$ 
20 else
21   return false

```

1.12 Parameter Sets and Instantiations

We provide eight parameter sets as shown in Table 1.3. Each parameter set name indicates the bit security against generic classical attacks, and whether the parameter set was designed to create relatively small signatures (**s**) or to have relatively fast signature generation (**f**). A parameter set consists of parameters for the one-time signature (n , **w**, z_b and **len**), **XMSS** and the **FlexTree** hypertree (h , **h** and d), the few-time signature (t , k and m_{MAX}) as well as instantiations for the cryptographic functions given in Table 1.4 and Table 1.5.

The functions described in Section 1.3 are instantiated differently in different algorithms

as shown in Table 1.4 and Table 1.5. The functions **PRF**, **F**, **H**, **H_{Root}** and **T_{len}** are heavily employed in key generation, signing and verification. To increase their efficiency, we pad the **PK.seed** input to one message block (i.e., 64 bytes) when the functions are instantiated with **SM3** (see Table 1.4). Since **SM3** is designed based on the Merkle-Damgård mode, this approach facilitates the reuse of intermediate state value after one-block computation. Finally, **ADRS^c** in Table 1.4 represents the compressed address derived from **ADRS**, where **ADRS^c.layerAddr** is the lowest byte of **ADRS.layerAddr** and **ADRS^c.addrType** is the lowest byte of **ADRS.addrType**, **ADRS^c.treeAddr** is the lowest 8 bytes of **ADRS.treeAddr**. Therefore, the size of **ADRS^c** is 22 bytes, which reduces the number of **SM3** compression function evaluations.

Warning: In this document, the cryptographic functions described in Section 1.3 are instantiated with hash functions or eXtended Output Functions (XOF) derived from **SM3**, which are provided in the **API_PKC.zip** file at https://niccs.org.cn/niccs/Download/API/API_PKC.zip. These functions are regarded as *ideal* primitives. For example, we assume that **XOF_{SM3}(msg, 8n)** provides 8n-bit (classical) security against (2nd) preimage attacks for all $n \in \mathbb{Z}_+$ even though this is not the case in reality. In particular, we ignore the security flaws due to the Merkle-Damgård construction of **SM3** [5]. These instantiations are made to conform to the rules specified by NICCS and to facilitate performance comparisons among submitted algorithms. *The actual security strength of these instances may not reach the specified security level due to dedicated attacks on SM3 and its derived functions.* Eventually, the cryptographic functions described in Section 1.3 should be instantiated with secure hash functions and XOFs.

Table 1.3: FlexTree parameter sets

Algorithm	n	h	d	t	k	m_{MAX}	$\mathbf{w}$	z_b	$\mathbf{len}$	Classical Sec. (Bits)	Quantum Sec. (Bits)	SK Size (Bytes)	PK Size (Bytes)	Sig Size (Bytes)
FlexTree-160s	20	67	9	42429	15	160	$[32] \times [64]^{25}$	5	26	160	80	80	40	9580
FlexTree-160f	20	67	17	4721	27	155	$[8] \times [16]^{39}$	1	40	160	80	80	40	18672
FlexTree-256s	32	67	10	81271	25	280	$[64]^{42}$	4	42	256	128	128	64	25420
FlexTree-256f	32	68	17	16900	35	270	$[8] \times [16]^{63}$	1	64	256	128	128	64	46856
FlexTree-384s	48	67	8	250491	35	398	$[16]^{94}$	8	94	384	192	192	96	60180
FlexTree-384f	48	64	12	71828	53	486	$[16]^3 \times [32]^{74}$	2	77	384	192	192	96	73396
FlexTree-512s	64	66	8	429271	48	560	$[32]^{101}$	7	101	512	256	256	128	94948
FlexTree-512f	64	66	11	181922	57	586	$[16] \times [32]^{101}$	3	102	512	256	256	128	117296

Table 1.4: Definition of the functions for FlexTree-160 and FlexTree-256

Function	Definition
$\mathbf{PRF}_{\text{MSG}}(\mathbf{SK}.\text{prf}, \text{optRand}, M)$	$\text{Trunc}_n(\text{SM3}(\mathbf{SK}.\text{prf} \parallel \text{optRand} \parallel M))$
$\mathbf{H}_{\text{MSG}}(\mathbf{R}, \mathbf{PK}.\text{seed}, \mathbf{PK}.\text{root}, M)$	$\text{XOF}_{\text{SM3}}(\mathbf{R} \parallel \mathbf{PK}.\text{seed} \parallel \mathbf{PK}.\text{root} \parallel M, 8m)$
$\mathbf{H}_{\text{PORS}}(\mathbf{R}, \text{md}, \text{ctr})$	$\text{XOF}_{\text{SM3}}(\text{SM3}(\mathbf{R} \parallel \text{md} \parallel \text{toByte}(0, 64 - 2n) \parallel \text{ctr}), 8 \cdot (n + \lceil \log_2 \binom{t}{k} / 8 \rceil))$
$\mathbf{H}_{\text{Root}}(\mathbf{PK}.\text{seed}, \text{ADRS}, M)$	$\text{Trunc}_n(\text{SM3}(\mathbf{PK}.\text{seed} \parallel \text{toByte}(0, 64 - n) \parallel \text{ADRS}^c \parallel M, 8n))$
$\mathbf{PRF}(\mathbf{PK}.\text{seed}, \mathbf{SK}.\text{seed}, \text{ADRS})$	$\text{Trunc}_n(\text{SM3}(\mathbf{PK}.\text{seed} \parallel \text{toByte}(0, 64 - n) \parallel \text{ADRS}^c \parallel \mathbf{SK}.\text{seed}, 8n))$
$\mathbf{F}(\mathbf{PK}.\text{seed}, \text{ADRS}, M_1)$	$\text{Trunc}_n(\text{SM3}(\mathbf{PK}.\text{seed} \parallel \text{toByte}(0, 64 - n) \parallel \text{ADRS}^c \parallel M_1, 8n))$
$\mathbf{H}(\mathbf{PK}.\text{seed}, \text{ADRS}, M_2)$	$\text{Trunc}_n(\text{SM3}(\mathbf{PK}.\text{seed} \parallel \text{toByte}(0, 64 - n) \parallel \text{ADRS}^c \parallel M_2, 8n))$
$\mathbf{T}_{\text{len}}(\mathbf{PK}.\text{seed}, \text{ADRS}, M)$	$\text{Trunc}_n(\text{SM3}(\mathbf{PK}.\text{seed} \parallel \text{toByte}(0, 64 - n) \parallel \text{ADRS}^c \parallel M, 8n))$

Table 1.5: Definition of the functions for FlexTree-384 and FlexTree-512

Function	Definition
$\mathbf{PRF}_{\text{MSG}}(\mathbf{SK.prf}, \text{optRand}, M)$	$\text{XOF}_{\text{SM3}}(\mathbf{SK.prf} \parallel \text{optRand} \parallel M, 8n)$
$\mathbf{H}_{\text{MSG}}(\mathbf{R}, \mathbf{PK.seed}, \mathbf{PK.root}, M)$	$\text{XOF}_{\text{SM3}}(\mathbf{R} \parallel \mathbf{PK.seed} \parallel \mathbf{PK.root} \parallel M, 8m)$
$\mathbf{H}_{\text{PORS}}(\mathbf{R}, \text{md}, \text{ctr})$	$\text{XOF}_{\text{SM3}}(\mathbf{R} \parallel \text{md} \parallel \text{ctr}, 8 \cdot (n + \lceil \log_2 \binom{t}{k} / 8 \rceil))$
$\mathbf{H}_{\text{Root}}(\mathbf{PK.seed}, \text{ADRS}, M)$	$\text{XOF}_{\text{SM3}}(\mathbf{PK.seed} \parallel \text{ADRS} \parallel M, 8n)$
$\mathbf{PRF}(\mathbf{PK.seed}, \mathbf{SK.seed}, \text{ADRS})$	$\text{XOF}_{\text{SM3}}(\mathbf{PK.seed} \parallel \text{ADRS} \parallel \mathbf{SK.seed}, 8n)$
$\mathbf{F}(\mathbf{PK.seed}, \text{ADRS}, M_1)$	$\text{XOF}_{\text{SM3}}(\mathbf{PK.seed} \parallel \text{ADRS} \parallel M_1, 8n)$
$\mathbf{H}(\mathbf{PK.seed}, \text{ADRS}, M_2)$	$\text{XOF}_{\text{SM3}}(\mathbf{PK.seed} \parallel \text{ADRS} \parallel M_2, 8n)$
$\mathbf{T}_{\text{len}}(\mathbf{PK.seed}, \text{ADRS}, M)$	$\text{XOF}_{\text{SM3}}(\mathbf{PK.seed} \parallel \text{ADRS} \parallel M, 8n)$

Chapter 2

Design Rationale

The SPHINCS⁺ framework provides the underlying architecture for modern quantum-resistant stateless hash-based signatures. Notable examples include the NIST standard SLH-DSA [6] and its recent variants [7, 3, 2, 4]. We extend the hypertree structure that underlies the SPHINCS⁺ framework by allowing trees of different heights to appear on different layers, and we plug generalized hash-based one-time signatures with chains of different lengths into the hypertree. Moreover, we optimize the few-time signatures based on the forced pruning technique. These optimizations lead to a larger design space, opening up the possibility for finer-grained trade-offs. This chapter gives the rationale behind the building blocks, including the inhomogeneous WOTS^c one-time signature and the PORS+FP few-time signature, as well as the FlexTree hypertree framework.

2.1 Notations and Preliminaries

Lemma 7. *Let $x_0, \dots, x_{d-1}$ be positive integers with $x_0 \leq x_1 \leq \dots \leq x_{d-1}$ and $x_0 + \dots + x_{d-1} = h$. If $|x_i - x_j| \leq 1$ for any $(i, j) \in [d] \times [d]$, then $x_0 = \dots = x_{d-k-1} = \lfloor \frac{h}{d} \rfloor$ and $x_{d-k} = \dots = x_{d-1} = \lceil \frac{h}{d} \rceil$, where $k = h - d \cdot \lfloor \frac{h}{d} \rfloor$.*

Proof. See [2]. □

Definition 2. *For given positive integers A and l , we define the notation*

$$\left\langle \frac{A}{l} \right\rangle = \min_{\substack{(x_0, \dots, x_{l-1}) \in \mathbb{Z}_+^l \\ x_0 + \dots + x_{l-1} = A}} \sum_{i=0}^{l-1} 2^{x_i}.$$

Lemma 8. *For given positive integers A and l ,*

$$\left\langle \frac{A}{l} \right\rangle = \min_{\substack{(x_0, \dots, x_{l-1}) \in \mathbb{Z}_+^l \\ x_0 + \dots + x_{l-1} = A}} \sum_{i=0}^{l-1} 2^{x_i} = (l - k) \cdot 2^{\lfloor A/l \rfloor} + k \cdot 2^{\lceil A/l \rceil},$$

where $k = A - l \cdot \lfloor \frac{A}{l} \rfloor$. That is, $\sum_{i=0}^{l-1} 2^{x_i}$ reaches its minimum when $(l - k)$ x_i 's take on the value $\lfloor \frac{A}{l} \rfloor$, and k x_i 's take on the value $\lceil \frac{A}{l} \rceil$.

Proof. See [2]. □

2.2 The Inhomogeneous WOTS^C One-time Signature

In our framework, messages signed by the one-time signature scheme are typically roots of hash trees. We therefore restrict the one-time signature message space to $\mathbb{B}^n$, the set of all n -byte strings. In addition, we present the construction using hash functions modeled as random oracles. We note, however, that modern hash-based one-time signatures rely on the so-called tweakable hash functions, and we expect all of our results to naturally carry over to that setting as well.

Let $H : \mathbb{F}_2^* \rightarrow \mathbb{F}_2^{8n}$ be a random oracle, z_b be a non-negative integer, $\text{len} \in \mathbb{Z}_+$, and $\mathbf{w} = (w_0, \dots, w_{\text{len}-1}) \in \mathbb{Z}_+^{\text{len}}$, such that $\log_2 w_i \in \mathbb{N}$, and $\log_2 w_0 + \dots + \log_2 w_{\text{len}-1} = 8n - z_b$. Then, we can construct a one-time signature scheme

$$\text{OTS}[H, \mathbb{F}_2^{8n}, z_b, \text{len}, \mathbf{w} = (w_0, \dots, w_{\text{len}-1})] = (\text{Gen}, \text{Sign}, \text{Ver})$$

as follows.

Construction 1. $\Phi = \text{OTS}[H, \mathbb{F}_2^{8n}, z_b, \text{len}, \mathbf{w} = (w_0, \dots, w_{\text{len}-1})] = (\text{Gen}, \text{Sign}, \text{Ver})$ is a one-time signature scheme with message space $\mathbb{F}_2^{8n}$ such that

- $(\text{sk}, \text{pk}) \leftarrow \text{Gen}(1^{8n})$:
 - For $i \in [\text{len}]$: $\text{sk}_i \xleftarrow{\$} \mathbb{F}_2^{8n}$, $\text{pk}_i \leftarrow H^{w_i-1}(\text{sk}_i)$
 - $\text{sk} := (\text{sk}_0, \dots, \text{sk}_{\text{len}-1})$, $\text{pk} := (\text{pk}_0, \dots, \text{pk}_{\text{len}-1})$
- $\sigma \leftarrow \text{Sign}(\text{sk}, m)$:
 - Find ctr such that $\text{leadingBits}(m', z_b) = 0^{z_b}$ and $\text{IntSeq}_{\mathbf{w}}(\text{trailingBits}(m', 8n - z_b)) \in \mathcal{C}_s \left(\prod_{j=0}^{l-1} [w_j] \right)$, where $m' \leftarrow H(\text{ctr} \parallel m)$ and $s = \left\lfloor \frac{(w_0-1) + \dots + (w_{\text{len}-1}-1)}{2} \right\rfloor$
 - $x := (x_0, \dots, x_{\text{len}-1}) \leftarrow \text{IntSeq}_{\mathbf{w}}(\text{trailingBits}(m', 8n - z_b))$
 - For $i \in [\text{len}]$: $\sigma_i \leftarrow H^{x_i}(\text{sk}_i)$
 - $\sigma := (\text{ctr}, \sigma_0, \dots, \sigma_{\text{len}-1}) \in \mathbb{N} \times [w_0] \times \dots \times [w_{\text{len}-1}]$
- $b \leftarrow \text{Ver}(\text{pk}, m, \sigma)$:
 - $(\text{ctr}, y_0, \dots, y_{\text{len}-1}) \leftarrow \sigma$
 - $m' \leftarrow H(\text{ctr} \parallel m)$
 - $x = (x_0, \dots, x_{\text{len}-1}) \leftarrow \text{IntSeq}_{\mathbf{w}}(\text{trailingBits}(m', 8n - z_b))$
 - For $i \in [\text{len}]$: $\text{pk}'_i \leftarrow H^{w_i-1-x_i}(y_i)$
 - $b := (\text{pk}_0 = \text{pk}'_0) \wedge \dots \wedge (\text{pk}_{\text{len}-1} = \text{pk}'_{\text{len}-1})$

Note that in existing one-time signatures employed in hash-based signature schemes [6, 3, 8, 7], $w_0 = \dots = w_{\text{len}-1} = w$, that is, the virtual hash chains produced by the schemes are of the same length. However, we emphasize that the security proofs for one-time signature schemes based on hash chains with the same length naturally carry over to the general case where $(w_0, \dots, w_{\text{len}-1})$ can take any value in $\mathbb{Z}_+^{\text{len}}$. Allowing chains with different lengths enlarges the design space, potentially leading to improved signature size, signing and verification performance.

Let $\mathfrak{C}_{\text{Sign}}(\Phi)$, $\mathfrak{C}_{\text{Ver}}(\Phi)$, and $\mathfrak{C}_{\text{Size}}(\Phi)$ denote the signing cost, verification cost, and signature size of Φ , respectively. According to Construction 1, we have the following estimation

$$\begin{cases} \mathfrak{C}_{\text{Size}}(\Phi) = n_{\text{ctr}}^{\text{OTS}} + \text{len} \cdot n \\ \mathfrak{C}_{\text{Sign}}(\Phi) = \frac{2^{z_b} \cdot \prod_{j=0}^{\text{len}-1} w_j}{|\mathcal{C}_s(\prod_{j=0}^{t-1} [w_j])|} + \left\lfloor \frac{(w_0-1) + \dots + (w_{\text{len}-1}-1)}{2} \right\rfloor + \text{len} \\ \mathfrak{C}_{\text{Ver}}(\Phi) = \left\lfloor \frac{(w_0-1) + \dots + (w_{\text{len}-1}-1)}{2} \right\rfloor \end{cases}, \quad (2.1)$$

where the unit for $\mathfrak{C}_{\text{Sign}}(\Psi)$ and $\mathfrak{C}_{\text{Ver}}(\Psi)$ is the number of hash function evaluations and the unit for $\mathfrak{C}_{\text{Size}}(\Psi)$ is the number of bytes.

2.3 The PORS+FP Few-time Signature Scheme

In this section, we describe the rationale behind the PORS+FP few-time signature scheme. Unlike one-time signatures, whose security guarantee applies only to a single use of each key, the security of a PORS+FP key degrades gradually as the number of signatures increases. PORS+FP improves upon the PORS few-time signature scheme [9, 10] with the forced-pruning technique, which rejects subsets whose pruned authentication set exceeds a fixed size.

2.3.1 PORS: PRNG to Obtain a Random Subset

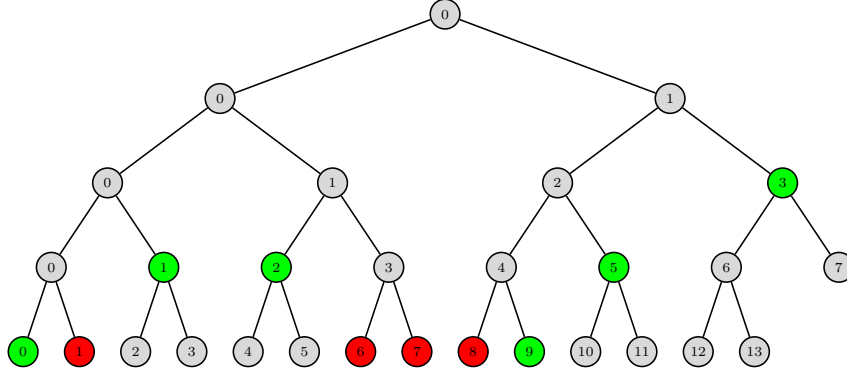
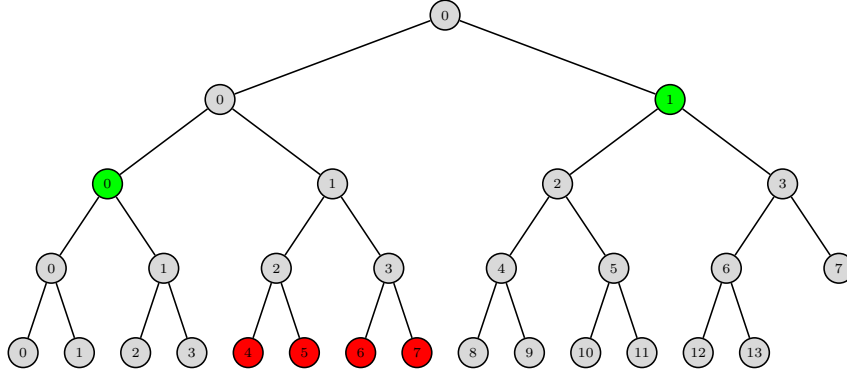
Let n , t , and k be positive integers with $k < t$. The PORS private key contains t secret values $x_0, x_1, \dots, x_{t-1}$ in $\mathbb{B}^n$. The public key is the root of the Merkle tree (called PORS tree) constructed with $y_0 = \mathbf{F}_0(x_0), \dots, y_{t-1} = \mathbf{F}_{t-1}(x_{t-1})$ being the leaf nodes, where $\{\mathbf{F}_i : \mathbb{F}_2^* \rightarrow \mathbb{B}^n\}$ is a family of hash functions. To sign a message msg , we hash msg with a hash function $\mathcal{H} : \mathbb{F}_2^* \rightarrow \binom{[t]}{k}$. Let $\mathcal{I} = \mathcal{H}(\text{msg})$. The signature on msg contains the secret values in $\{x_i : i \in \mathcal{I}\}$ and the authentication nodes for the leaf nodes in $\{y_i : i \in \mathcal{I}\}$. A trivial approach is to include the k authentication paths for $\{y_i : i \in \mathcal{I}\}$. However, there are a lot of redundancies in the resulting set of authentication nodes. For a given set $\mathcal{I}$, the *octopus* function (Algorithm 18) optimally prunes the k authentication paths and outputs the minimum set of authentication nodes with which it is sufficient to compute the root of the PORS tree based on the revealed elements in $\{x_i : i \in \mathcal{I}\}$. However, the number of nodes saved by pruning depends on the exact locations of the nodes in $\mathcal{I}$, and thus PORS has signatures of variable length. Two examples are given in Figure 2.1 and Figure 2.2, where for $\mathcal{I} = \{1, 6, 7, 8\}$ there are 6 nodes in $\text{octopus}(\mathcal{I})$ and for $\mathcal{I} = \{4, 5, 6, 7\}$ there are only 2 nodes in $\text{octopus}(\mathcal{I})$.

2.3.2 Limiting the Number of Authentication Nodes

In PORS+FP, for given (n, t, k) , to ensure a small signature size, we introduce a new parameter m_{MAX} . To sign a message msg , we repeatedly hash it with an $n_{\text{ctr}}^{\text{FTS}}$ -byte counter until we identify ctr such that

$$\text{octopus}(\mathcal{H}(\text{ctr} \parallel \text{msg})) \leq m_{\text{MAX}}. \quad (2.2)$$

Then the PORS+FP signature on msg contains the counter ctr , $x_i : i \in \mathcal{I}$, and the authentication nodes indexed by $\text{octopus}(\mathcal{I})$, where $\mathcal{I} = \mathcal{H}(\text{ctr} \parallel \text{msg})$. Note that the signature is zero padded to fixed length of $n_{\text{ctr}}^{\text{FTS}} + m_{\text{MAX}} \cdot n$ bytes.

Figure 2.1: The set of authentication nodes for $\mathcal{I} = \{1, 6, 7, 8\}$ Figure 2.2: The set of authentication nodes for $\mathcal{I} = \{4, 5, 6, 7\}$

Since the signing process is tasked with finding a counter `ctr` fulfilling Equation (2.2), it is important to determine the probability

$$\tau(t, k, m_{\text{MAX}}) = \Pr[\text{octopus}(\mathcal{I}) \leq m_{\text{MAX}}]$$

for a randomly generated k -element subset $\mathcal{I} \subseteq [t]$. Let $\theta(t, k)$ be the random variable denoting the number of elements in $\text{octopus}(\mathcal{I})$ for a uniformly random set $\mathcal{I} \in \binom{[t]}{k}$. Then,

$$\tau(t, k, m_{\text{MAX}}) = \sum_{m=0}^{m_{\text{MAX}}} \Pr[\theta(t, k) = m]. \quad (2.3)$$

We refer the reader to [4] for the method for computing $\Pr[\theta(t, k) = m]$.

2.4 The FlexTree Hypertree

The structure of the **FlexTree** virtual hypertree is described in Section 1.4. In our scheme, the height of the **XMSS** trees in layer i of the d -layer hypertree is h_i , while in existing hash-based signature schemes, $h_0 = \dots = h_{d-1}$. In short, we allow trees in different layers to be of different heights (but trees in the same layer are of the same height), which enlarges

the design space. Let $\mathfrak{C}_{\text{Sign}}(\Psi)$, $\mathfrak{C}_{\text{Ver}}(\Psi)$, and $\mathfrak{C}_{\text{Size}}(\Psi)$ denotes the signing cost, verification cost, and signature size of Ψ respectively for the hypertree signature described in Section 1.9. Then, we have the following estimation

$$\begin{cases} \mathfrak{C}_{\text{Size}}(\Psi) = d \cdot (n_{\text{ctr}}^{\text{OTS}} + \text{len} \cdot n) + h \cdot n \\ \mathfrak{C}_{\text{Sign}}(\Psi) = d \cdot \left(2^{z_b} + \left\lfloor \frac{(w_0-1) + \dots + (w_{l-1}-1)}{2} \right\rfloor + l \right) + (2^{h_0} - 1) + \dots + (2^{h_{d-1}} - 1) \\ \mathfrak{C}_{\text{Ver}}(\Psi) = d \cdot \left(\left\lceil \frac{(w_0-1) + \dots + (w_{l-1}-1)}{2} \right\rceil \right) + h \end{cases}, \quad (2.4)$$

where the unit for $\mathfrak{C}_{\text{Sign}}(\Psi)$ and $\mathfrak{C}_{\text{Ver}}(\Psi)$ is the number of hash function evaluations and the unit for $\mathfrak{C}_{\text{Size}}(\Psi)$ is the number of bytes. For a given value of $(n_{\text{ctr}}^{\text{OTS}}, n_{\text{ctr}}^{\text{FTS}}, \text{len}, \mathbf{w}, h, d, n, z_b)$, according to Lemma 8 and Equation (2.4), setting $h_0 = \dots = h_{k-1} = \lceil \frac{h}{d} \rceil$ and $h_k = \dots = h_{d-1} = \lfloor \frac{h}{d} \rfloor$ minimizes $\mathfrak{C}_{\text{Sign}}(\Psi)$, i.e., there is in essence a unique optimal value for $(h_0, \dots, h_{d-1})$. Note that we choose $h_0 = \dots = h_{k-1} = \lceil \frac{h}{d} \rceil$ rather than $h_0 = \dots = h_{k-1} = \lfloor \frac{h}{d} \rfloor$ since it leads to better performance for *FlexTree* key generation.

Chapter 3

Security Analysis

In this chapter, we analyze the security of **FlexTree**. Since **FlexTree** follows the general **SPHINCS⁺/SPHINCS⁺C** design and only optimizes its components, we do not give a new proof. Instead, in Section 3.1, we explain how the known security reductions carry over to our setting and note the additional point of generalizing the interleaved target subset resilience (ITSR) property for **PORS+FP**. Then, in Section 3.2, we analyze the concrete security of the proposed parameter sets.

3.1 Security Reduction

FlexTree does not require a fundamentally new proof. At the reduction level, it follows the **SPHINCS⁺/SPHINCS⁺C** proof strategy, with optimized one-time signature, hypertree, and few-time-signature components. Changes to the one-time signature and hypertree affect only bookkeeping, while replacing **FORS** by **PORS+FP** requires a generalization of ITSR at the few-time-signature layer.

For the one-time-signature layer, the proof is the same as for **WOTS^C** [3]. It only relies on the signed-digest encoding being injective and incomparable, which forces any valid forgery must move strictly earlier on at least one hash chain. This property also holds for our inhomogeneous **WOTS^C**; using chain lengths $(w_0, \dots, w_{l-1})$ therefore changes only the bookkeeping and concrete bound, not the reduction itself.

The same applies to the hypertree. In **SPHINCS⁺** [11], each XMSS layer signs the root of the layer below, allowing the verifier to reconstruct the public root. The proof only requires that each layer produces a valid XMSS signature; the actual heights of the layers do not affect the reduction. Replacing the uniform height h/d with layer heights $h_0, \dots, h_{d-1}$, where $\sum_{i=0}^{d-1} h_i = h$, changes only the number of authentication nodes per layer and leaving the reduction unchanged.

The main additional point in **FlexTree** is the few-time-signature component. In **SPHINCS⁺**, **H_{MSG}** must satisfy the ITSR property for the few-time-signature layer, which ensures that an adversary cannot forge a fresh message whose selected instance and leaves have all been revealed previously.

In **FlexTree**, a **PORS+FP** instance consists of a single tree, and **H_{PORS}** selects a k -element subset $I \subseteq [t]$. The ITSR property can be naturally generalized to this selection procedure $(R, \text{msg}, \text{ctr}) \mapsto (\text{idxTree}, \text{idxLeaf}, I)$, where $(\text{idxTree}, \text{idxLeaf})$ identifies the selected **PORS+FP** instance and I is the accepted **PORS+FP** subset. The generalized property requires that an adversary cannot find a fresh input whose subset I , *passing the pruning condition*,

is fully contained in previously revealed leaves for that instance.

Once this generalized subset-resilience property holds, the security of **FlexTree** follows directly from the existing reductions. The concrete impact is discussed in Section 3.2.

3.2 Concrete Security

We focus on our security analysis on the case where an attacker forges a **FlexTree** signature by forging a **PORS+FP** few-time signature for a unsigned message based on the information released as part of many **FlexTree** signatures issued previously without computing a (2nd) preimage of the underlying hash functions. Other strategies of generating a forgery are either covered by the existing analysis of the **SPHINCS⁺** framework or have a trivial complexity of about $\mathcal{O}(2^{8n})$.

Let q_{sig} be the number of signing queries overall. Then, the probability that a given **PORS+FP** instance is used exactly q times is about

$$\phi(q) = \binom{q_{\text{sig}}}{q} \left(1 - \frac{1}{2^h}\right)^{q_{\text{sig}}-q} \left(\frac{1}{2^h}\right)^q. \quad (3.1)$$

Let $p_{\text{Succ}}^{\text{FTS}}(q)$ be the probability that any given q signing queries of a specific **PORS+FP** instance can be used to successfully produce a forgery. Then, the probability $p_{\text{Succ}}(q_{\text{sig}})$ of successfully producing a **FlexTree** signature with q_{sig} signing queries is about

$$p_{\text{Succ}}(q_{\text{sig}}) = \sum_{q=0}^{q_{\text{sig}}} \binom{q_{\text{sig}}}{q} \left(1 - \frac{1}{2^h}\right)^{q_{\text{sig}}-q} \left(\frac{1}{2^h}\right)^q p_{\text{Succ}}^{\text{FTS}}(q). \quad (3.2)$$

Next, we show how to compute $p_{\text{Succ}}^{\text{FTS}}(q)$. Let the q signing queries lead to q **PORS+FP** signatures for $\text{md}_0, \dots, \text{md}_{q-1}$ with counter values $\text{ctr}_0, \dots, \text{ctr}_{q-1}$. Define

$$\mathcal{I}_j = \rho^{\binom{[t]}{k}} \left(\mathbf{H}_{\text{PORS}}(\text{ctr}_j, \text{md}_j) \bmod \binom{t}{k} \right). \quad 0 \leq j < q. \quad (3.3)$$

To produce a forgery for a randomly generated md , the adversary must find ctr such that $|\text{octopus}(\mathcal{J})| \leq m_{\text{MAX}}$ and $\mathcal{J} \subseteq \mathcal{I}_0 \cup \dots \cup \mathcal{I}_{q-1} \subseteq [t]$, where

$$\mathcal{J} = \rho^{\binom{[t]}{k}} \left(\mathbf{H}_{\text{PORS}}(\text{ctr}, \text{md}) \bmod \binom{t}{k} \right).$$

Consequently, the probability $p_{\text{Succ}}^{\text{FTS}}(q)$ can be evaluated as

$$\begin{aligned} p_{\text{Succ}}^{\text{FTS}}(q) &= \Pr[|\text{octopus}(\mathcal{J})| \leq m_{\text{MAX}} \wedge \mathcal{J} \subseteq \cup_{j=0}^{q-1} \mathcal{I}_j] \\ &= \Pr[|\text{octopus}(\mathcal{J})| \leq m_{\text{MAX}}] \cdot \Pr[\mathcal{J} \subseteq \cup_{j=0}^{q-1} \mathcal{I}_j \mid |\text{octopus}(\mathcal{J})| \leq m_{\text{MAX}}] \\ &= \tau(t, k, m_{\text{MAX}}) \cdot \Pr[\mathcal{J} \subseteq \cup_{j=0}^{q-1} \mathcal{I}_j \mid |\text{octopus}(\mathcal{J})| \leq m_{\text{MAX}}], \end{aligned} \quad (3.4)$$

where $\tau(t, k, m_{\text{MAX}}) = \Pr[|\text{octopus}(\mathcal{I})| \leq m_{\text{MAX}}]$ for a randomly generated k -element set $\mathcal{I} \in \binom{[t]}{k}$, which is exactly the probability that an iteration of the loop in Algorithm 19 (Lines 3 through 7) succeeds. Moreover,

$$\Pr[\mathcal{J} \subseteq \cup_{j=0}^{q-1} \mathcal{I}_j \mid |\text{octopus}(\mathcal{J})| \leq m_{\text{MAX}}] = \frac{\binom{|\cup_{j=0}^{q-1} \mathcal{I}_j|}{k}}{\tau(t, k, m_{\text{MAX}}) \cdot \binom{t}{k}} \leq \frac{\binom{k \cdot q}{k}}{\tau(t, k, m_{\text{MAX}}) \cdot \binom{t}{k}}. \quad (3.5)$$

Combining Equation (3.4) and Equation (3.5) gives

$$p_{\text{Succ}}^{\text{FTS}}(q) \leq \frac{\binom{k \cdot q}{k}}{\binom{t}{k}} = \frac{(k \cdot q)! (t - k)!}{(k \cdot q - k)! t!}. \quad (3.6)$$

Note that $p_{\text{Succ}}^{\text{FTS}}(q)$ is independent of $\tau(t, k, m_{\text{MAX}})$, and hence independent of m_{MAX} . Finally, Equation (3.2) and Equation (3.6) lead to the following formula

$$\min \left\{ 8n, -\log_2 \left(\sum_{q=0}^{q_{\text{sig}}} \binom{q_{\text{sig}}}{q} \left(1 - \frac{1}{2^h} \right)^{q_{\text{sig}} - q} \left(\frac{1}{2^h} \right)^q \frac{(k \cdot q)! (t - k)!}{(k \cdot q - k)! t!} \right) \right\}. \quad (3.7)$$

Chapter 4

Performance Analysis

In this chapter, we analyze the performance of our design, implement instances with the parameter sets defined in Section 1.12, and report on benchmark results.

4.1 Theoretical Performance Analysis

A **FlexTree** signature contains an n -byte randomizer, a $(n_{\text{ctr}}^{\text{FTS}} + k \cdot n + m_{\text{MAX}} \cdot n)$ -byte **PORS+FP** signature, d $(n_{\text{ctr}}^{\text{OTS}} + \text{len} \cdot n)$ -byte **WOTS^C** signatures, and h n -byte authentication nodes. Therefore, the total size of a **FlexTree** signature is

$$n + (n_{\text{ctr}}^{\text{FTS}} + k \cdot n + m_{\text{MAX}} \cdot n) + d \cdot (n_{\text{ctr}}^{\text{OTS}} + \text{len} \cdot n) + h \cdot n \quad (4.1)$$

bytes in length. Next, we analyze the performances of key generation, signing, and verification in terms of the number of hash function calls.

Key Generation Performance. The complexity of key generation is dominated by the complexity of computing the root of the top-level **XMSS** tree (Line 3 of Algorithm 22). Let $\mathfrak{C}_{\text{xmssNode}}(z)$ be the number of hash function evaluations of

$$\text{xmssNode}(\mathbf{SK}.\text{seed}, i, z, \mathbf{PK}.\text{seed}, \text{ADRS}).$$

Then, according to Algorithm 11, we have

$$\begin{cases} \mathfrak{C}_{\text{xmssNode}}(z) = 2\mathfrak{C}_{\text{xmssNode}}(z-1) + 1 \\ \mathfrak{C}_{\text{xmssNode}}(0) = \text{len} + (w_0 - 1) + \dots + (w_{\text{len}-1} - 1) + 1 = w_0 + \dots + w_{\text{len}-1} + 1 \end{cases} \quad (4.2)$$

Solving the recurrence given by Equation (4.2) gives $\mathfrak{C}_{\text{xmssNode}}(z) = 2^z \cdot \mathfrak{C}_{\text{xmssNode}}(0) + 2^z - 1$. Therefore, it requires about

$$(w_0 + \dots + w_{\text{len}-1} + 2) \cdot 2^{h_d-1} - 1 \quad (4.3)$$

hash function evaluations.

Signing Performance. The **FlexTree** signing process involves 1 hash function call to compute the initial **digest**, one **PORS+FP** signing operation, one call to $\text{pkFromSig}_{\text{PORS}}()$ to compute the **PORS** tree (Algorithm 20), d **WOTS^C** signing operations, the computation of h authentication nodes, and $d-1$ calls to $\text{pkFromSig}_{\text{XMSS}}()$ to compute the roots of the **XMSS**

trees (Algorithm 13), requiring h hash evaluations in total.

Let us first investigate the total complexity $\mathfrak{C}_{\text{FTS}}(t, k, m_{\text{MAX}})$ of one PORS+FP signing operation and one call to $\text{pkFromSig}_{\text{PORS}}()$. For $1 \leq k \leq t$, we denote $M(t, k)$ the random variable corresponding $\text{octopus}(I)$ when I is chosen uniformly from $\binom{[t]}{k}$. We further define

$$\tau(t, k, m_{\text{MAX}}) = \Pr[M(t, k) \leq m_{\text{MAX}}] = \sum_{m=0}^{m_{\text{MAX}}} \Pr[M(t, k) = m]. \quad (4.4)$$

One signing operation of PORS+FP involves about $\tau(t, k, m_{\text{MAX}})^{-1}$ calls to $\mathbf{H}_{\text{PORS}}$ (Line ?? of Algorithm 19). Then, the remaining part for generating the PORS+FP signature and computing the PORS+FP public key can be obtained by re-computing the necessary values associated with nodes of the PORS tree using at most $3t - 1$ hash evaluations, t hash evaluations for generating the t secret n -byte elements, t hash evaluations for computing the leaf nodes of the PORS+FP tree, and $t - 1$ hash evaluations for deriving the interior nodes according to Lemma 4. In short,

$$\mathfrak{C}_{\text{FTS}}(t, k, m_{\text{MAX}}) \leq \tau(t, k, m_{\text{MAX}})^{-1} + 3t - 1.$$

To generate one WOTS^c signature, it takes about

$$\mathfrak{C}_{\text{Sign}_{\text{OTS}}}(z_b, \mathbf{w}, \text{len}) = \frac{2^{z_b} \cdot \prod_{j=0}^{\text{len}-1} w_j}{\left| \mathcal{C}_s \left(\prod_{j=0}^{\text{len}-1} [w_j] \right) \right|} + \left\lfloor \frac{(w_0 - 1) + \dots + (w_{\text{len}-1} - 1)}{2} \right\rfloor + \text{len} \quad (4.5)$$

hash evaluations. Let $\mathfrak{C}_{\text{xmssAuth}}(h_i)$ be the number of hash evaluations required to compute the authentication nodes of an XMSS tree with height h_i . Then,

$$\begin{aligned} \mathfrak{C}_{\text{xmssAuth}}(h_i) &= \mathfrak{C}_{\text{xmssNode}}(0) + \dots + \mathfrak{C}_{\text{xmssNode}}(h_i - 1) \\ &= (\mathfrak{C}_{\text{xmssNode}}(0) \cdot 2^0 + 2^0 - 1) + \dots + (\mathfrak{C}_{\text{xmssNode}}(0) \cdot 2^{h_i-1} + 2^{h_i-1} - 1) \\ &= \mathfrak{C}_{\text{xmssNode}}(0) \cdot (2^{h_i} - 1) + (2^{h_i} - 1) - h_i. \end{aligned} \quad (4.6)$$

Consequently, the signing process takes about $1 + \mathfrak{C}_{\text{FTS}}(t, k, m_{\text{MAX}}) + d \cdot \mathfrak{C}_{\text{Sign}_{\text{OTS}}}(z_b, \mathbf{w}, \text{len}) + \sum_{i=0}^{d-1} \mathfrak{C}_{\text{xmssAuth}}(h_i) + h$, which is

$$1 + \mathfrak{C}_{\text{FTS}}(t, k, m_{\text{MAX}}) + d \cdot \mathfrak{C}_{\text{Sign}_{\text{OTS}}}(z_b, \mathbf{w}, \text{len}) + (\mathfrak{C}_{\text{xmssNode}}(0) + 1) \left(\left(\sum_{i=0}^{d-1} 2^{h_i} \right) - d \right).$$

hash evaluations to generate one FlexTree signature.

Verification Performance. Verification involves one hash call to compute the initial digest. One call to $\text{pkFromSig}_{\text{PORS}}()$, which requires $2k + m_{\text{MAX}}$ hash function calls. One call to $\text{verify}_{\text{HT}}()$, which requires $\left\lceil \frac{(w_0 - 1) + \dots + (w_{l-1} - 1)}{2} \right\rceil \cdot d + h$ hash function calls. It takes

$$1 + (2k + m_{\text{MAX}}) + \left(\left\lceil \frac{(w_0 - 1) + \dots + (w_{l-1} - 1)}{2} \right\rceil \cdot d + h \right) \quad (4.7)$$

hash evaluations to verify one FlexTree signature.

4.2 Implementations and Benchmarks

We implement the **FlexTree** instances for the defined parameter sets given in Section 1.12, the theoretical analysis are listed in Table 4.1, and the benchmark resulting cycle counts with AVX2-optimized are listed in Table 4.2. All of the results are obtained from running our codes compiled with `gcc-11.4.0-03-march=native-fomit-frame-pointer-flto` on a **Ubuntu 22.04** machine with **Intel Core i5-1135G7** CPU and 16GB RAM. It should be noted that, in the AVX2-optimized implementation, we did not optimize the hash function itself. Instead, we still use the reference hash implementation provided in the official `API_PKC.zip`.

Table 4.1: A theoretical analysis of the performance of the proposed signature algorithms

Scheme	PK Size (Bytes)	SK Size (Bytes)	Signature Size (Bytes)	Signing (#Hash call)	Verification (#Hash call)
FlexTree-160s	40	80	9580	2904774	7485
FlexTree-160f	40	80	18672	179947	5309
FlexTree-256s	64	128	25420	3204392	13628
FlexTree-256f	64	128	46856	322360	8501
FlexTree-384s	96	192	60180	5213804	6176
FlexTree-384f	96	192	73396	1448594	14697
FlexTree-512s	128	256	94948	9790642	13251
FlexTree-512f	128	256	117296	2836460	18070

Table 4.2: Benchmark results for the proposed signature algorithms

Scheme	Key Generation (cycles)	Signing (cycles)	Verification (cycles)
FlexTree-160s	383060529	5352451776	15968409
FlexTree-160f	9469379	357385366	10040305
FlexTree-256s	320854711	6035907933	25656773
FlexTree-256f	30299023	631705174	16529014
FlexTree-384s	2196682769	29385127065	35611026
FlexTree-384f	433413563	8211487016	82910827
FlexTree-512s	4598138941	55872250662	77376535
FlexTree-512f	1166834498	16427153023	103800840

4.3 Comparison with NIST Standard SLH-DSA

We compare the benchmark results of our **FlexTree** algorithm, whose underlying hash function is SHAKE, with NIST Standard **SLH-DSA** at the 256-bit security level. As can be observed, our scheme achieves a significant advantage in signature size(14.7% in **256s** and 6% in **256f**) and also provides faster signing performance. The only noticeable cost is the verification time of the **FlexTree-256s** instance. However, since verification is already relatively

fast in absolute terms, we consider this overhead acceptable. Moreover, the verification time of `FlexTree-256s` is faster than `SPHINCS+C-256s`.

Table 4.3: Performance comparison between `SPHINCS+-SHAKE-256*` and `FlexTree-SHAKE-256*`

Scheme	Key Generation (cycles)	Signing (cycles)	Verification (cycles)	Size (Bytes)
<code>SPHINCS⁺-SHAKE-256s</code>	54344637	639915462	1204508	29792
<code>FlexTree-SHAKE-256s</code>	32616202	619226941	3105477	25420
<code>SPHINCS⁺-SHAKE-256f</code>	3396239	67462413	2255661	49856
<code>FlexTree-SHAKE-256f</code>	3162953	66067627	2110293	46856

Chapter 5

Feature Overview

In this chapter, we summarize the new techniques employed in our design to reduce the cost and improve the performance of stateless hash-based signature schemes. We also discuss the advantages and limitations of the **FlexTree** hash-based signature scheme.

5.1 Techniques for Improving the SPHINCS⁺ Framework

The SPHINCS⁺ framework provides the underlying architecture for modern quantum-resistant stateless hash-based signatures. Notable examples include the NIST standard SLH-DSA and its recent variants such as SPHINCS- α and SPHINCS⁺C. We extend the hypertree structure that underlies the SPHINCS⁺ framework by allowing trees of different heights to appear on different layers, and we plug highly optimized few-time signatures (called **PORS+FP**) based on the forced-pruning technique and generalized hash-based one-time signatures with chains of different lengths into the hypertree. These techniques lead to a larger design space, opening up the possibility for finer-grained trade-offs.

The FlexTree Hypertree. We further extend the hypertree structure of SPHINCS⁺ and its variants by allowing the Merkle-like trees in different layers to have different heights. Let $h > 0$ be the total height and d be the number of layers such that $h = h_0 + \dots + h_{d-1}$, where $h_i > 0$ is the height of the trees in the i -th layer of the extended hypertree. For a given h , we show that in essence there is a unique choice of $(h_0, \dots, h_{d-1})$ that optimizes the performance.

One-time Signatures. We generalize the one-time signatures based on hash chains with chains of different lengths, and the resulting design is called *inhomogeneous* hash-chain-based one-time signatures. In particular, we propose an *inhomogeneous* hash-chain-based one-time signature scheme based on **WOTS^c**. Simply plugging the inhomogeneous one-time signatures into SPHINCS⁺ and its variants leads to size and signing and verification speed improvements.

Few-time Signatures. We explore using forced pruning in the few-time signature scheme used by state-less hash-based signature schemes to reduce the overall signature size. Prior work suggested similar ideas, but claimed that the improvement from forced pruning was small. We revisit this conclusion by performing a detailed theoretical analysis of forced pruning along with a more thorough exploration of its benefits. We show that forced-pruning can improve upon state-of-the-art stateless hash-based signature in all respects.

5.2 Advantages and Limitations

The advantages and limitations of **FlexTree** can be summarized as follows. It is widely regarded as one of the most conservative designs among post-quantum signature schemes, but it comes with inefficiencies in signature size and signing speed. Below, we examine its advantages and limitations in greater detail.

Minimal Security Assumptions. Unlike other post-quantum cryptographic schemes, including both signature and public-key encryption designs, **FlexTree** does not rely on any newly introduced intractability assumption. Its security is based solely on well-studied assumptions regarding the underlying hash function. While any practical signature scheme supporting arbitrary-length input requires a secure hash function, **FlexTree** offers an added layer of robustness: even a collision attack on the hash function does not compromise its security. This is a notable advantage in light of successful collision attacks against algorithms such as MD5 and SHA-1, especially considering that second-preimage resistance remains unbroken for MD5. Furthermore, the cryptographic community has developed a solid understanding of (exact) hash-function security, particularly following the recent SHA3 competition. This stands in contrast to the relatively novel mathematical problems employed in other branches of post-quantum cryptography. Although some of those problems have been known for a long time, accurately estimating the hardness of solving concrete instances remains far less mature.

Small key sizes. An additional advantage of **FlexTree** lies in its compact key sizes, particularly that of the public key. In many practical applications, certificates or certificate chains are routinely exchanged. The small public key size of **FlexTree** thus translates into tangible savings in bandwidth and storage in such deployment scenarios.

Signature size and speed. The primary drawback of **FlexTree** lies in its signing speed and signature size: it is not designed to compete as the fastest or most compact signature scheme. Nevertheless, the framework offers a wide range of possible trade-offs, allowing parameters to be adjusted as long as at least one of the two limitations – higher signing cost or larger signatures – can be accommodated. Moreover, **FlexTree** is fairly efficient in signature verification, a feature that makes it well-suited for application scenarios such as secure software updates, where verification performance is often more critical than signing speed.

Bibliography

- [1] NICCS. Call for Proposals for the Next-generation Public-Key Cryptographic Algorithms, 2025. https://www.niccs.org.cn/niccs/Notice/pc/content/content_1975896137741635584.html.
- [2] Zhen Qin and Siwei Sun. Extending the SPHINCS⁺ framework: Varying the tree heights and chain lengths. Cryptology ePrint Archive, Paper 2025/2236, 2025. <https://eprint.iacr.org/2025/2236>.
- [3] Andreas Hülsing, Mikhail A. Kudinov, Eyal Ronen, and Eylon Yogev. SPHINCS+C: compressing SPHINCS⁺ with (almost) no cost. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*, pages 1435–1453. IEEE, 2023.
- [4] Mehdi Abri and Jonathan Katz. Shorter hash-based signatures using forced pruning. CRYPTO 2026. <https://eprint.iacr.org/2025/2069>.
- [5] Ray A. Perlner, John Kelsey, and David A. Cooper. Breaking category five SPHINCS⁺ with SHA-256. In Jung Hee Cheon and Thomas Johansson, editors, *Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022, Virtual Event, September 28-30, 2022, Proceedings*, volume 13512 of *Lecture Notes in Computer Science*, pages 501–522. Springer, 2022.
- [6] NIST. Stateless Hash-Based Digital Signature Standard. Federal Information Processing Standards Publication, FIPS 205, 2024. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.205.pdf>.
- [7] Kaiyi Zhang, Hongrui Cui, and Yu Yu. Revisiting the Constant-Sum Winternitz One-Time Signature with Applications to SPHINCS⁺ and XMSS. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 455–483. Springer, 2023.
- [8] David McGrew, Michael Curcio, and Scott Fluhrer. Leighton-Micali Hash-Based Signatures. RFC 8554, 2019. <https://datatracker.ietf.org/doc/html/rfc8554>.
- [9] Jean-Philippe Aumasson and Guillaume Endignoux. Clarifying the subset-resilience problem. IACR ePrint 2017/909, 2017. <http://eprint.iacr.org/2017/909>.
- [10] Jean-Philippe Aumasson and Guillaume Endignoux. Improving stateless hash-based signatures. In Nigel P. Smart, editor, *Topics in Cryptology - CT-RSA 2018 - The*

- Cryptographers' Track at the RSA Conference 2018, San Francisco, CA, USA, April 16-20, 2018, Proceedings*, volume 10808 of *Lecture Notes in Computer Science*, pages 219–242. Springer, 2018.
- [11] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijnveld, and Peter Schwabe. The sphincs+ signature framework. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS '19*, page 2129–2146, New York, NY, USA, 2019. Association for Computing Machinery.